User Manual

for S32 ICU Driver

Document Number: UM11ICUASR4.4 Rev0000R4.0.2 Rev. 1.0

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

This User Manual describes NXP Semiconductor AUTOSAR ICU for S32. AUTOSAR ICU driver configuration parameters and deviations from the specification are described in ICU Driver chapter of this document. AUTOSAR ICU driver requirements and APIs are described in the AUTOSAR ICU driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
LPCMP	Low Power Comparator
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
EMIOS	Enhanced Modular IO Subsystem
FIFO	First In First Out
FTM	Flextimer Module
ICU	Input Capture Unit
ISR	Interrupt Service Routine
LSB	Least Signifigant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
OS	Operating System
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant
RAM	Random Access Memory
ROM	Read-only Memory
SIUL2	System Integration Unit Lite2
SWS	Software Specification
VLE	Variable Length Encoding
WKPU	Wakeup Unit
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of ICU Driver	AUTOSAR Release 4.4.0
2	S32G2 Reference Manual	Rev. 7, February 2023
3	S32G3 Reference Manual	Rev. 3, Feb 2023
4	S32R45 Reference Manual	Rev. 3, 12/2021

#	Title	Version
5	S32G2 Errata Document	Mask Set Errata for Mask 0P77B Rev. 3.0
6	S32G2 Errata Document	Mask Set Errata for Mask 1P77B Rev. 1.0
7	S32G3 Errata Document	Mask Set Errata for Mask 1P72B Rev. 2.1
8	S32R45 Errata Document	Mask Set Errata for Mask P57D, Rev. 2.0
9	S32G2 Data Sheet	Rev. 6, Dec 2022
10	S32G3 Data Sheet	Rev. 2, RC, Feb 2023
11	VR5510 Data Sheet	Rev. 5, April 2022
12	S32R45 Data Sheet	Rev. 3, RC — 11/2022

Chapter 3

Driver

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR 4.4 Rev0000ICU Driver Software Specification document (See Table Reference_list).

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See Table Reference List).

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The ICU Driver controls the input capture of the microcontroller. It provides the following features:

- High time / Low time measurement
- Duty Cycle measurement
- Period time measurement

- Edge detection and notification
- Edge counting (with or without hardware gating)
- Edge time stamping
- Wake-up interrupts

For signal edge detection, the edge detector of a capture compare unit or the interrupt controller for external events are used.

For signal measuring a capture timer and at least one capture register are needed. Also, only even channels (2∗n) can be used for signal measurements. This is because the channel after it (2∗n+1) is used internally by the ICU Driver

The FTM module of S32 supports period time measurement, edge detection and notification, edge counting and edge time stamping.

The SIUL2 and WKPU module of S32 supports edge detection and notification.

The ICU driver provides an optional API and configuration parameters for changing the base clock of the controlled hardware. A dual clock functionality is offered by switching between two configured values of the clock prescaler.

For each user configured channel, a symbolic name is generated by the Tresos Studio configuration tool. The name shall be consequently used in upper applications.

By default all channels offer interrupt handlers. For each channel not configured by the user in Tresos Studio configuration tool, the code for interrupt handling is removed based on a series of #ifdefs.

The RTD driver assures reentrancy (single core execution) for the APIs based on the following assumptions:

- the "called-again" API is for a different resource (hardware/logic channel);
- common variables/registers accessed with "rmw" are guarded by Exclusive Areas which need to be correctly implemented in RTE on user side;

3.3 Hardware Resources

The hardware resources configured by the Icu driver are next: **SIUL2**, **FTM** and **WKPU**. The SIUl2, FTM and WKPU input signal to microcontroller pin mapping can be done by using "IO_Signal_Description_and_Input_↩ multiplexing_tables.xls" from the Reference manual.

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR ICU Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented or out of scope for the ICU Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out of scope for the ICU driver.

Requirement	Status	Description	Notes
SWS_Icu_00150	N/S	The Icu module shall not check the integrity if several calls for the same ICU channel are used during runtime in different tasks or ISRs.	Rejection Reason: The requirement is violating safety because: The ICU149 is a safety integrity assumption for external environment, which shall be implemented for FTE; For GTE and NTE ICU149 has a role to increase availablity because the check will be supported by ICU driver; see also 00149
SWS_Icu_00380	N/S	These requirements are not applicable to this specification.	Not a requirement
SWS_Icu_91002	N/S	This function disables the notification of a channel.Tags: atp.Status=draft	Description specified as draft is not clear. Should be re-assesed on next ASR version
SWS_Icu_91003	N/S	This function enables the notification on the given channel.Tags: atp.↩ Status=draft	Description specified as draft is not clear. Should be re-assesed on next ASR version

3.5 Driver limitations

- Limited NMI functionality present in EB Tresos and S32 Design Studio but not validated.
- Limited validation of S32 Design Studio components for High Level Driver (HLD) and Low Level Drivers (IPL).
- Limited FTM functionality present in IPL.
- Only support the HLD and IPL separated.

3.6 Driver usage and configuration tips

In this chapter, the extra features from our drivers that are not described in the AutoSAR standard are detailed

3.6.1 Icu with DMA feature

In order to speed up data transfer, the Direct Memory Access feature can be used. The DMA feature can be used only in Timestamp mode and is done with the help of the Mcl driver to transfer timestamp data from a Icu input match dirrectly in the Timestamp buffer. Thus, this is an example of Peripheral to Memory data transfer

and it's very usefull for avoiding the interrupt overhead. In order to have this feature, the user should check the IcuDMAChannelEnable checkbox for DMA in the Icu channel. This will be selectable only for a channel in Timestamp mode. Also, the user has to configure a Mcl channel with a Mcl Dma Transfer Completion User Notification named <IcuChannelName>_MclDmaTransferCompletionNotif for the respective Icu Channel. for example, for Icu Channel 4, the notification should be named IcuChannel_4_MclDmaTransferCompletionNotif. Apart from that, the user should configure the DMA instance that allows transfer from that ICU modules. The DMA channel mapping from the RM shows the sources mapping for each peripheral in the DMA channel mapping chappter. This can also be observed in the Mcl plugin in the list of DMA sources. Other fields from the DMA's TCD do not require explicit configuration since they are specific to the ICU's FTM module.

3.6.2 Dual Clock Feature

In order to allow dinamic change of the driver working frequency, the ICU driver has the Dual Clock Feature. The IcuEnableDualClockMode from IcuAutosarExt should be enabled in order to have this feature active. Afterwards, the Prescaler_Alternate parameter allows setting a different prescaler for each module. These parameters will be changed when calling the function call Icu_SetClockMode.
Icu_SetClockMode may be called only after Icu_Init is called and when IcuEnableDualClockMode is checked. Our suggested ussage of this API is to call it when the driver is in a lower power state but still in active use.

3.6.3 WKPU channel selection

From WKPU peripheral perspective the input channels are counted from 0 to 31 (Note: only 24 hardware channels are available for usage)

```
0 to 22 channels are routed to external pins that are named as: WKPU0 to WKPU22.
31 channel is internally routed to RTC module (see reference manual)
```

Duty cycle measurement using BOTH EDGES measurement limitation explained.
When this type of measurement is used the limitation consist in considering if the measured pulsWidth is always HIGH TIME.
For example, in the next figure it can be seen that for the same signal, pulseWidth will have the same value, even for the second one measurement where the active time can be considered the LOW TIME as well.
In the I) and the II) case the result will be the same dutyCycle = 25%, even the signal will be consider with LOW TIME as the active time.
Pros for this measurement:
\t -> the measurement will be done with no wait time
Cons for this measurement:
\t -> the result will consider the HIGH TIME as the active time for all the measurements with BOTH EDG↩ ES measurement, but there is an workaround if it is known that the active time is the LOW TIME dutyCycle = 100 - currentDutyCycle(e.g in our case if we consider the LOW TIME as the active time dutyCycle=100-25=75%)

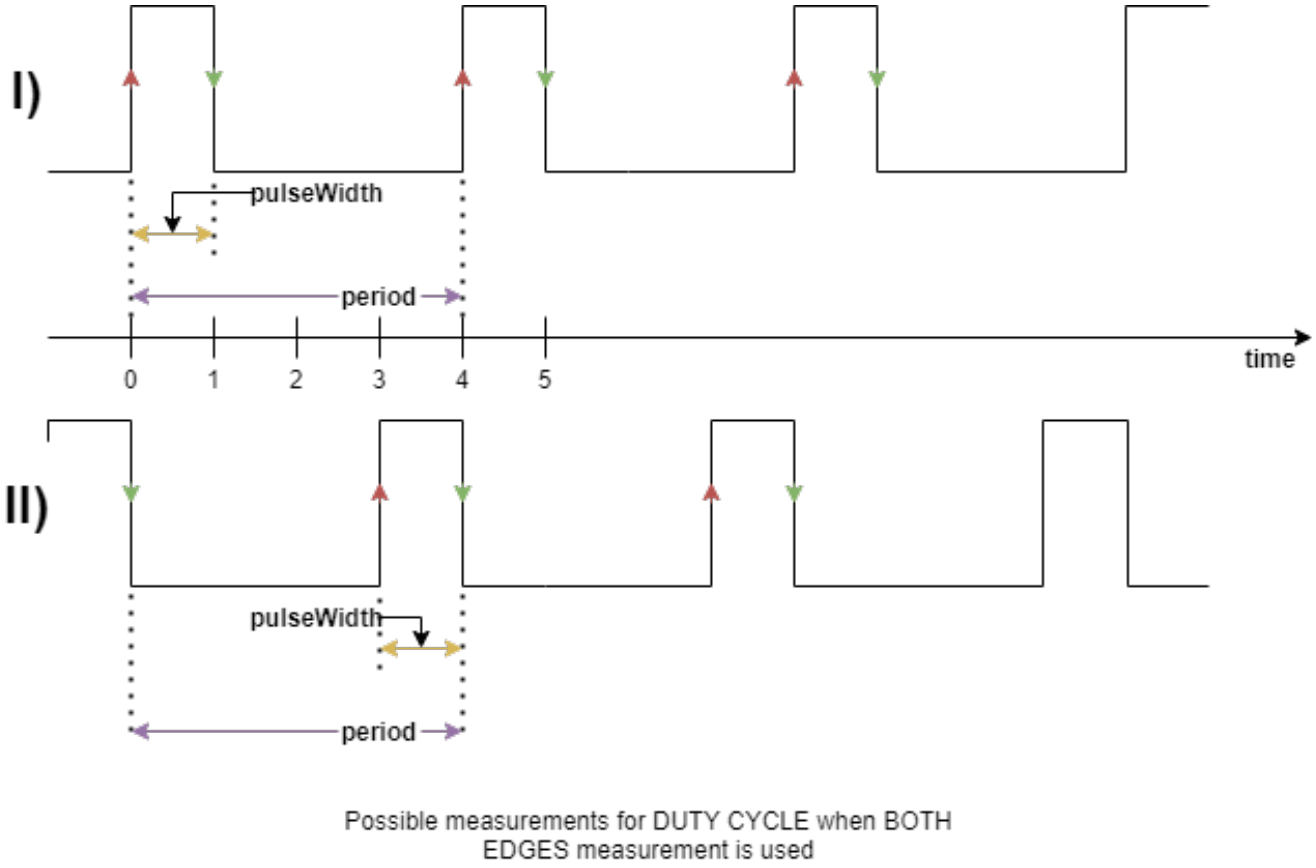

Figure 3.1 Possible measurements for DUTY CYCLE when BOTH EDGES measurement is used

3.7 Runtime Errors

The driver does not generate any DEM runtime errors.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

#define <Mip>Conf_<Container_ShortName>_<Container_ID>

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module Icu
  - Container IcuConfigSet
    - Parameter IcuMaxChannel
    - Container IcuChannel
      - Parameter IcuChannelId
      - Parameter IcuDMAChannelEnable
      - Parameter IcuDefaultStartEdge
      - Parameter IcuMeasurementMode
      - Parameter IcuOverflowNotification
      - Parameter IcuWakeupCapability
      - Reference IcuChannelEcucPartitionRef
      - Reference IcuChannelRef
      - Reference IcuDMAChannelRef
      - Container IcuSignalEdgeDetection
      - Parameter IcuSignalNotification
      - Container IcuSignalMeasurement
      - Parameter IcuSignalMeasurementProperty
      - Container IcuTimestampMeasurement
      - Parameter IcuTimestampMeasurementProperty
      - Parameter IcuTimestampNotification
      - Container IcuWakeup
      - Reference IcuChannelWakeupInfo
    - Container IcuFtm
      - Parameter IcuFtmModule
      - Parameter IcuFtmClockSource
      - Parameter IcuFtmPrescaler
      - Parameter IcuFtmPrescalerAlternate
      - Parameter IcuFtmDebugMode
      - Parameter IcuFtmModValue
      - Container IcuFtmChannels

- Parameter IcuFtmChannel
- Parameter IcuFtmFilter

* Container IcuSiul2
  - Parameter IcuSiul2Instance
  - Parameter IcuEXT_ISR_InterruptFilterClockPrescaler
  - Parameter IcuEXT_ISR_AlternateInterruptFilterClockPrescaler
  - Container IcuSiul2Channels
  - Parameter IcuSiul2Channel
  - Parameter Icu_EXT_ISR_IFERDigitalFilter
  - Parameter Icu_EXT_ISR_IFMCDigitalFilter
* Container IcuWkpu
  - Parameter WkpuPullOverride
  - Container IcuWkpuChannels
  - Parameter IcuWkpuChannel
  - Parameter WakeupBootModeSelect
  - Parameter Icu_EXT_ISR_WIFERDigitalFilter
  - Parameter WkpuPadPullValue
  - Container IcuWkpuNMIConfiguration
  - Parameter NMICoreSource
  - Parameter DestinationSourceSelect
  - Parameter WakeupRequestEnable
  - Parameter FilterEnable
  - Parameter NMIEdgeEvents
  - Parameter LockRegister
* Container IcuHwInterruptConfigList
  - Parameter IcuIsrHwId
  - Parameter IcuIsrEnable

– Container IcuGeneral

* Parameter IcuDevErrorDetect
* Parameter IcuReportWakeupSource
* Parameter IcuEnableUserModeSupport
* Parameter IcuMulticoreSupport
* Reference IcuEcucPartitionRef
* Reference IcuKernelEcucPartitionRef

– Container IcuAutosarExt

* Parameter IcuEnableDualClockMode
* Parameter IcuOverflowNotificationApi
* Parameter IcuGetInputLevelApi
* Parameter IcuGetCaptureRegisterValueApi
* Parameter IcuWkpuStandbyWakeupSupport

– Container IcuOptionalApis

* Parameter IcuDeInitApi
* Parameter IcuDisableWakeupApi
* Parameter IcuEdgeCountApi

        * Parameter IcuEnableWakeupApi
        * Parameter IcuGetDutyCycleValuesApi
        * Parameter IcuGetInputStateApi
        * Parameter IcuGetTimeElapsedApi
        * Parameter IcuGetVersionInfoApi
        * Parameter IcuSetModeApi
        * Parameter IcuSignalMeasurementApi
        * Parameter IcuTimestampApi
        * Parameter IcuWakeupFunctionalityApi
        * Parameter IcuEdgeDetectApi
    - Container CommonPublishedInformation
        * Parameter ArReleaseMajorVersion
        * Parameter ArReleaseMinorVersion
        * Parameter ArReleaseRevisionVersion
        * Parameter ModuleId
        * Parameter SwMajorVersion
        * Parameter SwMinorVersion
        * Parameter SwPatchVersion
        * Parameter VendorApiInfix
        * Parameter VendorId

4.1 Module Icu

Configuration of the Icu (Input Capture Unit) module

Included containers:

- IcuConfigSet
- IcuGeneral
- IcuAutosarExt
- IcuOptionalApis
- CommonPublishedInformation

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container IcuConfigSet

This container is the base for a multiple configuration set

Included subcontainers:

- IcuChannel
- IcuFtm
- IcuSiul2
- IcuWkpu
- IcuHwInterruptConfigList

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter IcuMaxChannel

The value for the IcuMaxChannel must match with the number of IcuChannel configured

For calculating the correct value use the CALC button.

Note: Total number of configured channels should be same across all IcuConfigSets.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	3
max	68
min	1

4.4 Container IcuChannel

Configuration of an individual ICU channel.

Included subcontainers:

- IcuSignalEdgeDetection
- IcuSignalMeasurement
- IcuTimestampMeasurement
- IcuWakeup

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.5 Parameter IcuChannelId

Channel Id of the ICU channel.

This value will be assigned to the symbolic name derived of the IcuChannel container short name.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	1
max	67
min	0

4.6 Parameter IcuDMAChannelEnable

IcuDMAChannelEnable indicates if the corresponding channel will use DMA for measurement

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.7 Parameter IcuDefaultStartEdge

Configures the default-activation-edge which shall be used for this channel

if there was no activation-edge configured by the call of service Icu_SetActivationCondition().

In case the Measurement Mode is "IcuSignalMeasurement" and the properties "DutyCycle" or "Period" are set, the edge configured here is used as Default Period Start Edge.

Implementation Type: Icu_ActivationType

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	ICU_RISING_EDGE
literals	['ICU_BOTH_EDGES', 'ICU_FALLING_EDGE', 'ICU_RISING_EDGE']

4.8 Parameter IcuMeasurementMode

Configures the measurement mode of this channel.

User should enable optional parameters with respect to the selected IcuMeasurementMode.

Implementation Type: Icu_MeasurementModeType

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	ICU_MODE_SIGNAL_EDGE_DETECT
literals	['ICU_MODE_EDGE_COUNTER', 'ICU_MODE_SIGNAL_EDGE_DETE↩ CT', 'ICU_MODE_SIGNAL_MEASUREMENT', 'ICU_MODE_TIMESTA↩ MP']

4.9 Parameter IcuOverflowNotification

Icu Overflow Notification Handler

In order to activate this field you have to:

enable IcuOverflowNotificationApi,

choose one of the modes:

ICU_MODE_EDGE_COUNTER,

ICU_MODE_SIGNAL_MEASUREMENT,

ICU_MODE_TIMESTAMP

to enable overflow detection on the internal counter

Note:

Due to hardware implementattion, the Icu Overflow Notification

is not syncronous with the event for ICU_MODE_SIGNAL_MEASUREMENT

and ICU_MODE_TIMESTAMP modes.

The notification will be triggered when measurement completes (for ICU_MODE_SIGNAL_MEASUREMENT)

or the next timestamp event occurs (for ICU_MODE_TIMESTAMP).

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.10 Parameter IcuWakeupCapability

Information about the wakeup-capability of this channel.

true: Channel is wakeup capable.

false: Channel is not wakeup capable.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.11 Reference IcuChannelEcucPartitionRef

Maps a ICU channel to zero or multiple ECUC partitions to limit the access to this channel group.

The ECUC partitions referenced are a subset of the ECUC partitions where the ICU driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.12 Reference IcuChannelRef

Select the ICU hw channel on which the functionality of the current ICU channel will be implemented

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/TS_T40D11M40I2R0/Icu/IcuConfigSet/IcuFtm/IcuFtmChannels', '/TS_↩ T40D11M40I2R0/Icu/IcuConfigSet/IcuWkpu/IcuWkpuChannels', '/TS_T40↩ D11M40I2R0/Icu/IcuConfigSet/IcuWkpu/IcuWkpuNMIConfiguration', '/TS_↩ T40D11M40I2R0/Icu/IcuConfigSet/IcuSiul2/IcuSiul2Channels']

4.13 Reference IcuDMAChannelRef

Icu DMA Channel Reference

Reference to the DMA Channel configure for the Request

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/TS_T40D11M40I2R0/Mcl/MclConfig/dmaLogicChannel_Type']

4.14 Container IcuSignalEdgeDetection

This container contains the configuration (parameters) in case the measurement mode is "IcuSignalEdgeDetection"

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.15 Parameter IcuSignalNotification

Notification function for signal notification

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.16 Container IcuSignalMeasurement

This container contains the configuration (parameters) in case the measurement mode is "IcuSignalMeasurement"

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.17 Parameter IcuSignalMeasurementProperty

Configures the property that could be measured in case the mode is "IcuSignalMeasurement".

This property can not be changed during runtime.

Followings are measurement mode

ICU_DUTY_CYCLE

ICU_HIGH_TIME

ICU_LOW_TIME

ICU_PERIOD_TIME

Implementation Type: Icu_SignalMeasurementPropertyType

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	ICU_DUTY_CYCLE
literals	['ICU_DUTY_CYCLE', 'ICU_HIGH_TIME', 'ICU_LOW_TIME', 'ICU_P↩ ERIOD_TIME']

4.18 Container IcuTimestampMeasurement

This container contains the configuration (parameters) in case the measurement mode is "IcuTimestamp"

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.19 Parameter IcuTimestampMeasurementProperty

Configures the handling of the buffer in case the mode is "Timestamp"

Following type of buffer inplemented in current implementation.

ICU_CIRCULAR_BUFFER.

ICU_LINEAR_BUFFER.

Implementation Type: Icu_TimestampBufferType

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	ICU_LINEAR_BUFFER
literals	['ICU_CIRCULAR_BUFFER', 'ICU_LINEAR_BUFFER']

4.20 Parameter IcuTimestampNotification

Notification function if the number of requested timestamps(Notification interval > 0) are acquired

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.21 Container IcuWakeup

This container contains the configuration (parameters) needed to configure a wakeup capable channel

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.22 Reference IcuChannelWakeupInfo

If the wakeup-capability is true the wakeup source referenced is transmitted to the ECU State Manager (EcuM) .

Implementation Type: reference to EcuM_WakeupSourceType

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuM/EcuMConfiguration/EcuMCommon↩ Configuration/EcuMWakeupSource

4.23 Container IcuFtm

Configuration of a FTM module available on the platfom.

Included subcontainers:

- IcuFtmChannels

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.24 Parameter IcuFtmModule

Select which hardware instance of FTM to use.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	1
min	0

4.25 Parameter IcuFtmClockSource

Select origin of clock source used by current instance of FTM.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	SYSTEM_CLOCK
literals	['SYSTEM_CLOCK', 'EXTERNAL_CLOCK', 'FIXED_FREQUENCY_CLOCK']

4.26 Parameter IcuFtmPrescaler

Prescaler used by current FTM instance.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1
max	128
min	0

4.27 Parameter IcuFtmPrescalerAlternate

Set an alternante prescaler for FTM instance. With this option the frequency can be changed at runtime.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	128
max	128
min	0

4.28 Parameter IcuFtmDebugMode

Mode 0: FTM counter stopped, CH(n)F bit can be set, FTM channels in functional mode, writes to MOD,CNTIN and C(n)V registers bypass the register buffers.

Mode 1: FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are forced to their safe value , writes to MOD,CNTIN and C(n)V registers bypass the register buffers.

Mode 2: FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are frozen when chip enters in BDM mode, writes to MOD, CNTIN and C(n)V registers bypass the register buffers.

Mode 3: FTM counter in functional mode, CH(n)F bit can be set,FTM channels in functional mode, writes to MOD,CNTIN and C(n)V registers is in fully functional mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MODE_0
literals	['MODE_0', 'MODE_1', 'MODE_2', 'MODE_3']

4.29 Parameter IcuFtmModValue

Maxim value of counter for current FTM instance. This parameter will define the maxim pulse period which can be measured.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	65535
min	0

4.30 Container IcuFtmChannels

List of Ftm channels available on the platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.31 Parameter IcuFtmChannel

Select a hardware channel of the current FTM instance to configure.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	5
min	0

4.32 Parameter IcuFtmFilter

Input Capture Filter Control: Selects the filter value for the channel input. The filter is disabled when the value is zero.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	15
min	0

4.33 Container IcuSiul2

Configuration of a Siul2 module available on the platfom.

Included subcontainers:

- IcuSiul2Channels

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	2
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.34 Parameter IcuSiul2Instance

Select the hardware instance of SIUL2.

Only support interrupt in SIUL2_1 instance. SIUL2_0 has no interrupt.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1
max	1
min	0

4.35 Parameter IcuEXT_ISR_InterruptFilterClockPrescaler

Configure the clock prescaler which is used to select the clock for all digital filter counters in the SIUL2NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	15
min	0

4.36 Parameter IcuEXT_ISR_AlternateInterruptFilterClockPrescaler

Configure the clock alternate prescaler which is used to select the clock for all digital filter counters in the SIUL2. This is only available when Dual Clock mode is activated.

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	15
min	0

4.37 Container IcuSiul2Channels

List of Siul2 Channels on flatform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.38 Parameter IcuSiul2Channel

Selects one of the Siul2 channels available on the platform.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	31
min	0

4.39 Parameter Icu_EXT_ISR_IFERDigitalFilter

Enable external digital filter counter on the interrupt pads to filter out glitches on the inputs

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.40 Parameter Icu_EXT_ISR_IFMCDigitalFilter

Maximum Interrupt Filter Counter setting.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	15
min	0

4.41 Container IcuWkpu

Configuration of a Wkpu module available on the platfom.

Included subcontainers:

- IcuWkpuChannels
- IcuWkpuNMIConfiguration

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.42 Parameter WkpuPullOverride

This field selects one of two methods to control the WKUP pins.

The SIUL2 or WKPU field configuration controls the pull value of wake-up pads.

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.43 Container IcuWkpuChannels

List of Wkpu modules available on the platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.44 Parameter IcuWkpuChannel

Selects one of the Wkpu channels available on the platform.

Wake-up sources 0 to 22 : map to the pads with the corresponding WKPU signal and I/O supplies

Wake-up source 31 : map to the internal RTC module

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	31
min	0

4.45 Parameter WakeupBootModeSelect

Wakeup Boot Mode Select is used by the boot software to select between a short boot or a full boot. Application software should write 0 or 1

for each wakeup event. User must ensure that the selected Boot mode matches the corresponding wakeup source.

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.46 Parameter Icu_EXT_ISR_WIFERDigitalFilter

Wakeup/Interrupt Filter Enable is used to enable an analog filter on the corresponding interrupt pads to filter out glitches on the inputs.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.47 Parameter WkpuPadPullValue

The PULL value of WKPU pads are controlled by WKPU configuration

HIGH_IMPEDANCE

PULL_UP

PULL_DOWN

Note

This feature is only supported Wakeup sources 0 to 22 which is mapped to the pads with the corresponding WKPUx signal and I/O supplies.

This is enabled by choosing WKUP Pull Override Enable in WKPU Modules configuration.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	HIGH_IMPEDANCE
literals	['HIGH_IMPEDANCE', 'PULL_UP', 'PULL_DOWN']

4.48 Container IcuWkpuNMIConfiguration

The WKPU NMI configuration for each core.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.49 Parameter NMICoreSource

Selects one of the supported cores for the NMI.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	0
min	0

4.50 Parameter DestinationSourceSelect

NMI Destination Source Select

As wakeup does not support another interrupt than NMI, the destination source select signal bits are reserved and always retain their reset value. This means no other request other than NMI can be generated.

NMI_NON_MASK_INT: Non-maskable interrupt

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NMI_NON_MASK_INT
literals	['NMI_NON_MASK_INT']

4.51 Parameter WakeupRequestEnable

System wakeup requests from the corresponding NIF0 field

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.52 Parameter FilterEnable

Enable analog glitch filter on the NMI pad input..

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.53 Parameter NMIEdgeEvents

Configures the NMI Edge Events which shall be used for this NMI

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	RISING_EDGE
literals	['RISING_EDGE', 'FALLING_EDGE', 'BOTH_EDGES']

4.54 Parameter LockRegister

Locks the configuration for the NMI until it is unlocked by a system reset or STANDBY0 mode exit.

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.55 Container IcuHwInterruptConfigList

List of HW interrupts available for the entire platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.56 Parameter IcuIsrHwId

Id of the HW interrupt service routine available platform wide and usable by ICU module.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
defaultValue	FTM_0_CH_1
literals	['FTM_0_CH_0', 'FTM_0_CH_1', 'FTM_0_CH_2', 'FTM_0_CH_3', 'FTM_0_CH_4', 'FTM_0_CH_5', 'FTM_1_CH_0', 'FTM_1_CH_1', 'FTM_1_CH_2', 'FTM_1_CH_3', 'FTM_1_CH_4', 'FTM_1_CH_5', 'IRQ_CH_0', 'IRQ_CH_1', 'IRQ_CH_2', 'IRQ_CH_3', 'IRQ_CH_4', 'IRQ_CH_5', 'IRQ_CH_6', 'IRQ_CH_7', 'IRQ_CH_8', 'IRQ_CH_9', 'IRQ_CH_10', 'IRQ_CH_11', 'IRQ_CH_12', 'IRQ_CH_13', 'IRQ_CH_14', 'IRQ_CH_15', 'IRQ_CH_16', 'IRQ_CH_17', 'IRQ_CH_18', 'IRQ_CH_19', 'IRQ_CH_20', 'IRQ_CH_21', 'IRQ_CH_22', 'IRQ_CH_23', 'IRQ_CH_24', 'IRQ_CH_25', 'IRQ_CH_26', 'IRQ_CH_27', 'IRQ_CH_28', 'IRQ_CH_29', 'IRQ_CH_30', 'IRQ_CH_31', 'WKPU_CH_0', 'WKPU_CH_1', 'WKPU_CH_2', 'WKPU_CH_3', 'WKPU_CH_4', 'WKPU_CH_5', 'WKPU_CH_6', 'WKPU_CH_7', 'WKPU_CH_8', 'WKPU_CH_9', 'WKPU_CH_10', 'WKPU_CH_11', 'WKPU_CH_12', 'WKPU_CH_13', 'WKPU_CH_14', 'WKPU_CH_15', 'WKPU_CH_16', 'WKPU_CH_17', 'WKPU_CH_18', 'WKPU_CH_19', 'WKPU_CH_20', 'WKPU_CH_21', 'WKPU_CH_22', 'WKPU_CH_31']

4.57 Parameter IcuIsrEnable

Status of the HW Interrupt (true - Interrupt shall be enable platform wide; false - Interrupt shall be disabled platform wide.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.58 Container IcuGeneral

Configuration of general ICU parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.59 Parameter IcuDevErrorDetect

Switches the Development Error Detection and Notification on or off.

true: Enabled.

false: Disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.60 Parameter IcuReportWakeupSource

Switch for enabling Wakeup source reporting.

true: Report Wakeup source.

false: Do not report Wakeup source.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.61 Parameter IcuEnableUserModeSupport

When this parameter is enabled, the Icu module will adapt to run from User Mode, with the following measures:

a) configuring REG_PROT for SIUL2 IP so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1

b) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.

for more information, please see chapter 5.7 User Mode Support in IM

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.62 Parameter IcuMulticoreSupport

When this parameter is enabled, the ICU module will adapt to run with Multicore:

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.63 Reference IcuEcucPartitionRef

Maps the ICU driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Depending on the addressed timer resource the interfaces operate as follows.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.64 Reference IcuKernelEcucPartitionRef

Maps the ICU kernel to zero or one ECUC partitions to assign the driver kernel to a certain core.

The ECUC partition referenced is a subset of the ECUC partitions where the ICU driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.65 Container IcuAutosarExt

Enabling the settings of this section will configure the driver in a mode not compliant with AUTOSAR requirements.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.66 Parameter IcuEnableDualClockMode

Enables prescaler settings at mode transition.

true: Enabled.

false: Disabled.

Note: This feature is not required by Autosar.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.67 Parameter IcuOverflowNotificationApi

Add / removes Overflow Notification functionality.

Enabling IcuOverflowNotificationApi overflow events will not be treated as errors and a Notification Handler can be provided.

If this optional API is not enabled, overflow events will trigger DET Report Error.

Note:

Due to hardware implementattion, the Icu Overflow Notification is not syncronous with

the event for ICU_MODE_SIGNAL_MEASUREMENT and ICU_MODE_TIMESTAMP modes. The notification

will be triggered when measurement completes (for ICU_MODE_SIGNAL_MEASUREMENT) or the

next timestamp event occurs (for ICU_MODE_TIMESTAMP).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.68 Parameter IcuGetInputLevelApi

Add / removes Icu_GetInputLevel API from the code.

This function returns Input pin state.

Note: This feature is not required by Autosar.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.69 Parameter IcuGetCaptureRegisterValueApi

Adds / removes service Icu_GetCaptureRegisterValue from the code.

This function returns value of Capture register for the mesurement channel or timestamp mode channel which is called by the user.

It's enabled when IcuTimestampApi or IcuSignalMeasurementApi is true.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.70 Parameter IcuWkpuStandbyWakeupSupport

Icu_Init() will not clear the wakeup flags (WISR register) if it is already set during init.

Note: This feature is not required by Autosar.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.71 Container IcuOptionalApis

This container contains all configuration switches for configuring optional API services of the ICU driver.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.72 Parameter IcuDeInitApi

Adds / removes the service Icu_DeInit() from the code.

true: Icu_DeInit() can be used.

false: Icu_DeInit() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.73 Parameter IcuDisableWakeupApi

Adds / removes the service Icu_DisableWakeup() from the code.

true: Icu_DisableWakeup() can be used.

false: Icu_DisableWakeup() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.74 Parameter IcuEdgeCountApi

Adds / removes all services related to the edge counting
functionality as listed below, from the code: Icu_ResetEdgeCount(),
Icu_EnableEdgeCount(), Icu_DisableEdgeCount(), Icu_GetEdgeNumbers().

true: The services listed above can be used.

false: The services listed above can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.75 Parameter IcuEnableWakeupApi

Adds / removes the service Icu_EnableWakeup() from the code.

true: Icu_EnableWakeup() can be used.

false: Icu_EnableWakeup() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.76 Parameter IcuGetDutyCycleValuesApi

Adds / removes the service Icu_GetDutyCycleValues() from the code.

true: Icu_GetDutyCycleValues() can be used.

false: Icu_GetDutyCycleValues() can not be used.

Note: If IcuSignalMeasurementApi == OFF this switch is shall also be set to OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.77 Parameter IcuGetInputStateApi

Adds / removes the service Icu_GetInputState() from the code.

true: Icu_GetInputState() can be used.

false: Icu_GetInputState() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.78 Parameter IcuGetTimeElapsedApi

Adds / removes the service Icu_GetTimeElapsed() from the code.

true: Icu_GetTimeElapsed() can be used.

false: Icu_GetTimeElapsed() can not be used.

Note: If IcuSignalMeasurementApi == OFF this switch is shall also be set to OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.79 Parameter IcuGetVersionInfoApi

Adds / removes the service Icu_GetVersionInfo() from the code.

true: Icu_GetVersionInfo() can be used.

false: Icu_GetVersionInfo() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.80 Parameter IcuSetModeApi

Adds / removes the service Icu_SetMode() from the code.

true: Icu_SetMode() can be used.

false: Icu_SetMode() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.81 Parameter IcuSignalMeasurementApi

Adds / removes the services Icu_StartSignalMeasurement() and Icu_StopSignalMeasurement() from the code.

true: Icu_StartSignalMeasurement() and Icu_StopSignalMeasurement() can be used.

false: Icu_StartSignalMeasurement() and Icu_StopSignalMeasurement() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.82 Parameter IcuTimestampApi

Adds / removes all services related to the timestamping functionality as listed below from the code:
Icu_StartTimestamp(), Icu_StopTimestamp(), Icu_GetTimestampIndex().

true: The services listed above can be used.

false: The services listed above can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.83 Parameter IcuWakeupFunctionalityApi

Adds / removes the service Icu_CheckWakeup() from the code.

true: Icu_CheckWakeup() can be used.

false: Icu_CheckWakeup() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.84 Parameter IcuEdgeDetectApi

Adds / removes the services Icu_EnableEdgeDetection() and Icu_DisableEdgeDetection() from the code.

true: Icu_EnableEdgeDetection() and Icu_DisableEdgeDetection() can be used.

false: Icu_EnableEdgeDetection() and Icu_DisableEdgeDetection() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.85 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.86 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.87 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.88 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.89 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	122
max	122
min	122

4.90 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION

Property	Value
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.91 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.92 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	2

Property	Value
max	2
min	2

4.93 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires

that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the

implementation specific name is generated as follows: <ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	

4.94 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

None.

This chapter describes the Tresos configuration plug-in for the *driver* Driver. The most of the parameters are described below.

Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Chapter 6

Module Documentation

6.1 WKPU IPL

6.1.1 Detailed Description WKPU HW module.

The Wakeup Unit (WKPU) supports external sources that can generate interrupts or wakeup events and external source(s) that can cause non-maskable interrupt request(s) or wakeup event(s).
In addition, it combines its wakeup events with those generated from other wakeup sources to supply a single wakeup to the system.

Wakeup Unit Configurations
The 'Wakeup Unit (WKPU)' provides a configurable low-power wakeup capability to the device from multiple configurable asynchronous wakeup events. The wakeup unit on the device supports 4 internal sources and 60 external sources that can generate interrupts or wakeup events. It also supports a non-maskable interrupt input.

External signal description

- The WKPU has signal inputs that can be used as external interrupt sources in normal run mode or as system wakeup sources in certain power down modes.

NMI configuration

The Wakeup Unit (WKPU) supports one external source that can cause non-maskable interrupts to on-chip cores and wakeup events to the system. Herein there are two application cores indicated, CM7_0 and CM7_1. In the event of any wakeup (internal or external), the WKPU initiates the recovery of the device and feeds this interrupt to the core(s) depending on the configurations.

Data Structures

- struct Wkpu_Ip_NmiCfgType
  NMI configuration structure. *More...*
- struct Wkpu_Ip_ChannelConfigType
  WKPU interrupt configuration structure. *More...*
- struct Wkpu_Ip_IrqConfigType
  Wkpu IP specific configuration structure type. *More...*
- struct Wkpu_Ip_State
  WKPU IP state structure. *More...*

Types Reference

- typedef void(* Wkpu_Ip_NotifyType) (void)
  The notification functions shall have no parameters and no return value.

Enum Reference

- enum Wkpu_Ip_NmiDestSrcType
  NMI destination source.
- enum Wkpu_Ip_EdgeType
  Edge event.
- enum Wkpu_Ip_StatusType
  Wkpu_Ip_StatusType.
- enum Wkpu_Ip_PullType
- enum Wkpu_Ip_CoreType

Function Reference

- void Wkpu_Ip_SetSleepMode (uint8 instance, uint8 hwChannel)
  ICU driver function that sets WKPU channel to SLEEP mode.
- void Wkpu_Ip_SetNormalMode (uint8 instance, uint8 hwChannel)
  Icu driver function that sets WKPU channel to NORMAL mode.
- void Wkpu_Ip_EnableInterrupt (uint8 instance, uint8 hwChannel)
  Enable the interrupt request and wakeup generation.
- void Wkpu_Ip_DisableInterrupt (uint8 instance, uint8 hwChannel)
  ICU driver function that disables the interrupt of a WKPU channel.
- Wkpu_Ip_StatusType Wkpu_Ip_Init (uint8 instance, const Wkpu_Ip_IrqConfigType *userConfig)
  Icu driver function that initializes WKPU channels.
- Wkpu_Ip_StatusType Wkpu_Ip_DeInit (uint8 instance)
  ICU driver function that resets WKPU configuration.
- void Wkpu_Ip_SetActivationCondition (uint8 instance, uint8 hwChannel, Wkpu_Ip_EdgeType edge)
  ICU driver function that sets activation condition of WKPU channel.
- boolean Wkpu_Ip_GetInputState (uint8 instance, uint8 hwChannel)
  ICU driver function that gets the input state of WKPU channel.
- void Wkpu_Ip_EnableNotification (uint8 hwChannel)
  Driver function Enable Notification for timestamp.
- void Wkpu_Ip_DisableNotification (uint8 hwChannel)
  Driver function Disable Notification for timestamp.
- INTERRUPT_FUNC void WKPU_EXT_IRQ_SINGLE_ISR (void)
  Interrupt handler for WKPU channels: 0 to 31.
- void Wkpu_Ip_SetUserAccessAllowed (uint32 GprBaseAddr)
  Wkpu_Ip_SetUserAccessAllowed.

6.1.2 Data Structure Documentation

6.1.2.1 struct Wkpu_Ip_NmiCfgType

NMI configuration structure.

Definition at line 167 of file Wkpu_Ip_Types.h.

Data Fields

Type	Name	Description
Wkpu_Ip_CoreType	core	WKPU core source.
Wkpu_Ip_NmiDestSrcType	destination	NMI destination source.
boolean	wkpReqEn	NMI request enable.
boolean	filterEn	NMI filter enable.
Wkpu_Ip_EdgeType	edgeEvent	NMI edge events.
boolean	lockEn	NMI configuration lock register.

6.1.2.2 struct Wkpu_Ip_ChannelConfigType

WKPU interrupt configuration structure.

Definition at line 183 of file Wkpu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	hwChannel	The WKPU hardware channel.
boolean	filterEn	WKPU/interrupt filter enable.
Wkpu_Ip_PullType	pullValue	The pull value of wake-up pads.
Wkpu_Ip_EdgeType	edgeEvent	WKPU/interrupt edge events.
boolean	bootSel	WKPU select boot mode. If true, Long boot execution by boot software after wakeup from STANDBY mode. otherwise Short boot execution by boot software after wakeup from STANDBY mode.
Wkpu_Ip_CallbackType	callback	Pointer to the callback function.
Wkpu_Ip_NotifyType	WkpuChannelNotification	The notification functions shall have no parameters and no return value.
uint16	callbackParam	The logic channel for which callback is set.

6.1.2.3 struct Wkpu_Ip_IrqConfigType

Wkpu IP specific configuration structure type.

Definition at line 203 of file Wkpu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	numNMIChannels	Number of channels in NMI configuration.
Wkpu_Ip_NmiCfgType(*	pNMIChannelsConfig)[]	Pointer to channels configuration.

Data Fields

Type	Name	Description
uint8	numChannels	Number of channels in configuration.
boolean	pullOverrideEn	Selects one of two methods to control the WKUP pins.
const Wkpu_Ip_ChannelConfigType(*	pChannelsConfig)[]	Pointer to channels configuration.

6.1.2.4 struct Wkpu_Ip_State

WKPU IP state structure.

This structure is used by the IPL driver for internal logic. The content is populated at initialization time.

Definition at line 226 of file Wkpu_Ip_Types.h.

Data Fields

Type	Name	Description
boolean	chInit	Initialization state.
Wkpu_Ip_CallbackType	callback	Pointer to the callback function.
Wkpu_Ip_NotifyType	WkpuChannelNotification	The notification functions for SIGNAL_EDGE_DETECT mode.
uint16	callbackParam	The logic channel for which callback is set.
boolean	notificationEnable	Store the initialization state that determines whether Notifications are enabled.

6.1.3 Types Reference

6.1.3.1 Wkpu_Ip_NotifyType

```
typedef void(* Wkpu_Ip_NotifyType) (void)
```

The notification functions shall have no parameters and no return value.

Definition at line 153 of file Wkpu_Ip_Types.h.

6.1.4 Enum Reference

6.1.4.1 Wkpu_Ip_NmiDestSrcType

```
enum Wkpu_Ip_NmiDestSrcType
```

NMI destination source.

Enumerator

WKPU_IP_NMI_NONE	No NMI, critical interrupt, or machine check request generated.
WKPU_IP_NMI_NON_MASK_INT	Non-maskable interrupt.

Definition at line 102 of file Wkpu_Ip_Types.h.

6.1.4.2 Wkpu_Ip_EdgeType

enum Wkpu_Ip_EdgeType

Edge event.

Enumerator

WKPU_IP_NONE_EDGE	None event.
WKPU_IP_RISING_EDGE	Rising edge event.
WKPU_IP_FALLING_EDGE	Falling edge event.
WKPU_IP_BOTH_EDGES	Both rising and falling edge event.

Definition at line 120 of file Wkpu_Ip_Types.h.

6.1.4.3 Wkpu_Ip_StatusType

enum Wkpu_Ip_StatusType

Wkpu_Ip_StatusType.

This indicates the operation success or fail

Enumerator

WKPU_IP_SUCCESS	Status for success operation return.
WKPU_IP_ERROR	General error return status.

Definition at line 132 of file Wkpu_Ip_Types.h.

6.1.4.4 Wkpu_Ip_PullType

```
enum Wkpu_Ip_PullType
```

Enumerator

WKPU_IP_HIGH_IMPEDANCE	The WKPU field configuration will controls the pull value of wake-up pads: High-impedance
WKPU_IP_PULL_UP	The WKPU field configuration will controls the pull value of wake-up pads: Pull up
WKPU_IP_PULL_DOWN	The WKPU field configuration will controls the pull value of wake-up pads: Pull down

Definition at line 141 of file Wkpu_Ip_Types.h.

6.1.4.5 Wkpu_Ip_CoreType

```
enum Wkpu_Ip_CoreType
```

Enumerator

WKPU_CORE0	Core 0
WKPU_CORE1	Core 1
WKPU_CORE2	Core 2

Definition at line 156 of file Wkpu_Ip_Types.h.

6.1.5 Function Reference

6.1.5.1 Wkpu_Ip_SetSleepMode()

```
void Wkpu_Ip_SetSleepMode (
            uint8 instance,
            uint8 hwChannel )
```

ICU driver function that sets WKPU channel to SLEEP mode.

This function enables the interrupt for WKPU channel if wakeup is enabled for the channel.

Parameters

in	*instance*	Hardware instance of WKPU used.
in	*hwChannel*	Hardware channel of WKPU used.

Returns

void

6.1.5.2 Wkpu_Ip_SetNormalMode()

```
void Wkpu_Ip_SetNormalMode (
             uint8 instance,
             uint8 hwChannel )
```

Icu driver function that sets WKPU channel to NORMAL mode.

This function enables the interrupt for WKPU channel if Notifcation is enabled for the channel.

Parameters

in	*instance*	Hardware instance of WKPU used.
in	*hwChannel*	Hardware channel of WKPU used.

Returns

void

6.1.5.3 Wkpu_Ip_EnableInterrupt()

```
void Wkpu_Ip_EnableInterrupt (
             uint8 instance,
             uint8 hwChannel )
```

Enable the interrupt request and wakeup generation.

This function setup generation of interrupt and wakeup generation.

Parameters

in	*instance*	Hardware instance of WKPU used.
in	*hwChannel*	Hardware channel of WKPU used.

Returns

void

6.1.5.4 Wkpu_Ip_DisableInterrupt()

```
void Wkpu_Ip_DisableInterrupt (
            uint8 instance,
            uint8 hwChannel )
```

ICU driver function that disables the interrupt of a WKPU channel.

This function disables WKPU Channel Interrupt.

Parameters

in	*instance*	Hardware instance of WKPU used.
in	*hwChannel*	Hardware channel of WKPU used.

Returns

void

6.1.5.5 Wkpu_Ip_Init()

```
Wkpu_Ip_StatusType Wkpu_Ip_Init (
            uint8 instance,
            const Wkpu_Ip_IrqConfigType * userConfig )
```

Icu driver function that initializes WKPU channels.

This function:

- Sets Interrupt Filter Enable Register
- Sets Wakeup/Interrupt Pull-up Enable Register
- Sets Activation Condition

Parameters

in	*instance*	Hardware instance of WKPU used.
in	*userConfig*	- Pointer to array of with channels configuration.

Returns

void

6.1.5.6 Wkpu_Ip_DeInit()

```
Wkpu_Ip_StatusType Wkpu_Ip_DeInit (
            uint8 instance )
```

ICU driver function that resets WKPU configuration.

This function:

- Disables IRQ Interrupt
- Clears Wakeup/Interrupt Filter Enable Register
- Clears Wakeup/Interrupt Pull-up Enable Register
- Clears edge event enable registers
- Clear Interrupt Filter Enable Register

Parameters

in	*instance*	Hardware instance of WKPU used.

Returns

void

6.1.5.7 Wkpu_Ip_SetActivationCondition()

```
void Wkpu_Ip_SetActivationCondition (
            uint8 instance,
            uint8 hwChannel,
            Wkpu_Ip_EdgeType edge )
```

ICU driver function that sets activation condition of WKPU channel.

This function enables the requested activation condition(rising, falling or both edges) for corresponding WKPU channels.

Parameters

in	*instance*	Hardware instance of WKPU used.
in	*hwChannel*	Hardware channel of WKPU used.
in	*edge*	Edge type for activation.

Returns

void

6.1.5.8 Wkpu_Ip_GetInputState()

```
boolean Wkpu_Ip_GetInputState (
            uint8 instance,
            uint8 hwChannel )
```

ICU driver function that gets the input state of WKPU channel.

This function:

- Checks if interrupt flags for corresponding WKPU channel is set then it clears the interrupt flag and returns the value as TRUE.

Parameters

in	*instance*	Hardware instance of WKPU used.
in	*hwChannel*	Hardware channel of WKPU used.

Returns

boolean

- TRUE - if channel is active
- FALSE - If channel is in idle

6.1.5.9 Wkpu_Ip_EnableNotification()

```
void Wkpu_Ip_EnableNotification (
            uint8 hwChannel )
```

Driver function Enable Notification for timestamp.

6.1.5.10 Wkpu_Ip_DisableNotification()

```
void Wkpu_Ip_DisableNotification (
            uint8 hwChannel )
```

Driver function Disable Notification for timestamp.

6.1.5.11 WKPU_EXT_IRQ_SINGLE_ISR()

```
INTERRUPT_FUNC void WKPU_EXT_IRQ_SINGLE_ISR (
            void  )
```

Interrupt handler for WKPU channels: 0 to 31.

Process the interrupt of WKPU channels 0 to 31 for platforms with only one interrupt line.

Remarks

This will be defined only if any of WKPU channels 0 to 31 are configured.

6.1.5.12 Wkpu_Ip_SetUserAccessAllowed()

```
void Wkpu_Ip_SetUserAccessAllowed (
            uint32 GprBaseAddr )
```

Wkpu_Ip_SetUserAccessAllowed.

This function is called externally by OS Application

Parameters

in	*GprBaseAddr*	- The base address of GPR module.

6.2 FTM IPL

6.2.1 Detailed Description FTM HW module.

FTM IP layer hardware module.

Data Structures

- struct Ftm_Icu_Ip_DutyCycleType

  Structure that contains ICU Duty cycle parameters. It contains the values needed for calculating duty cycles i.e Period time value and active time value. More...
- struct Ftm_Icu_Ip_ChannelConfigType

  FlexTimer driver Input capture parameters for each channel. More...
- struct Ftm_Icu_Ip_InstanceConfigType

  FTM IP layer module configuration. More...
- struct Ftm_Icu_Ip_ConfigType

  FTM driver input capture parameters. More...
- struct Ftm_Icu_Ip_ChStateType

  This structure is used by the IPL driver for internal logic. More...
- struct Ftm_Icu_Ip_InstStateType

  This structure is used by the IPL driver for internal logic. More...

Macros

- #define CHAN0_IDX

  Channel number for CHAN0.
- #define CHAN1_IDX

  Channel number for CHAN1.
- #define CHAN2_IDX

  Channel number for CHAN2.
- #define CHAN3_IDX

  Channel number for CHAN3.
- #define CHAN4_IDX

  Channel number for CHAN4.
- #define FTM_MAX_VAL_COUNTER

  Maximum value of FTM counter.
- #define FTM_COMBINE_COMBINEx_SHIFT(u8ChannelIdx)

  Set flag COMINEx for specified pair: 0, 1, 2.

Types Reference

- typedef void(* Ftm_Icu_Ip_NotifyType) (void)

  The notification functions shall have no parameters and no return value.
- typedef void(* Ftm_Icu_Ip_CallbackType) (uint16 callbackParam1, boolean callbackParam2)

  Callback type for each channel.
- typedef void(* Ftm_Icu_Ip_LogicChState) (uint16 logicChannel, uint8 mask, boolean set)

  Callback type for each channel.

Enum Reference

- enum Ftm_Icu_Ip_ClockSourceType

  FTM clock source selection.
- enum Ftm_Icu_Ip_DebugModeType

  FTM debug modes of operation.
- enum Ftm_Icu_Ip_EdgeType

  Activation condition for the measurement - selecting edge type.
- enum Ftm_Icu_Ip_ModeType

  Operation mode for ICU driver.
- enum Ftm_Icu_Ip_SubModeType

  Enable/disable DMA support for timestamp.
- enum Ftm_Icu_Ip_MeasType

  Type of operation for signal measurement.
- enum Ftm_Icu_Ip_LevelType

  Enumeration used for returning the level of input pin.
- enum Ftm_Icu_Ip_ClockModeType

  Definition of prescaler type (Normal or Alternate)
- enum Ftm_Icu_Ip_StatusType

  Generic error codes.
- enum Ftm_Icu_Ip_TimestampBufferType

  Type of operation for timestamp.

Function Reference

- Ftm_Icu_Ip_StatusType Ftm_Icu_Ip_Init (uint8 instance, const Ftm_Icu_Ip_ConfigType *userConfig)
- Ftm_Icu_Ip_StatusType Ftm_Icu_Ip_DeInit (uint8 instance)

  Disables input capture mode and clears FTM timer configuration.
- void Ftm_Icu_Ip_SetSleepMode (uint8 instance, uint8 hwChannel)

  Driver function sets FTM hardware channel into SLEEP mode.
- void Ftm_Icu_Ip_SetNormalMode (uint8 instance, uint8 hwChannel)

  Driver function sets FTM hardware channel into NORMAL mode.
- uint32 Ftm_Icu_Ip_GetCaptureRegisterValue (uint8 instance, uint8 hwChannel)

  Capture the value of counter register for a specified channel.
- void Ftm_Icu_Ip_SetActivationCondition (uint8 instance, uint8 hwChannel, Ftm_Icu_Ip_EdgeType activation)
- void Ftm_Icu_Ip_StartTimestamp (uint8 instance, uint8 hwChannel, Ftm_Icu_ValueType *bufferPtr, uint16 bufferSize, uint16 notifyInterval)

  FTM IP layer function which starts timestamp measure mode.
- void Ftm_Icu_Ip_StopTimestamp (uint8 instance, uint8 hwchannel)

  FTM IP layer function which stops timestamp measure mode for a given instance and channel.
- uint16 Ftm_Icu_Ip_GetTimestampIndex (uint8 instance, uint8 hwChannel)

  This function reads the timestamp index of the given channel.
- void Ftm_Icu_Ip_EnableEdgeDetection (uint8 instance, uint8 hwChannel)

  FTM IP layer function which enable edge detection measure mode for a given instance and channel.
- void Ftm_Icu_Ip_DisableEdgeDetection (uint8 instance, uint8 hwChannel)

FTM IP layer function which disable edge detection measure mode for a given instance and channel.

- void Ftm_Icu_Ip_ResetEdgeCount (uint8 instance, uint8 hwchannel)

  FTM IP layer function which reset edge count measure mode for a given instance and channel.
- Ftm_Icu_Ip_StatusType Ftm_Icu_Ip_EnableEdgeCount (uint8 instance, uint8 hwChannel)

  FTM IP layer function which enable edge count measure mode for a given instance and channel.
- void Ftm_Icu_Ip_DisableEdgeCount (uint8 instance, uint8 hwChannel)

  FTM IP layer function which disable edge count measure mode for a given instance and channel.
- uint16 Ftm_Icu_Ip_GetEdgeNumbers (uint8 instance, uint8 hwChannel)

  FTM IP layer function which gets the number of edges for a given instance and channel.
- void Ftm_Icu_Ip_StartSignalMeasurement (uint8 instance, uint8 hwchannel)

  FTM IP layer function which starts signal measurement mode for a given instance and channel.
- void Ftm_Icu_Ip_StopSignalMeasurement (uint8 instance, uint8 hwchannel)

  FTM IP layer function which stops signal measurement mode for a given instance and channel.
- uint16 Ftm_Icu_Ip_GetTimeElapsed (uint8 instance, uint8 hwChannel)

  This function reads the elapsed Signal Low, High or Period Time for the given channel.
- void Ftm_Icu_Ip_GetDutyCycleValues (uint8 instance, uint8 hwChannel, Ftm_Icu_Ip_DutyCycleType *dutyCycleValues)

  This function reads the coherent active time and period time for the given ICU Channel.
- void Ftm_Icu_Ip_SetPrescaler (uint8 instance, Ftm_Icu_Ip_ClockModeType selectPrescaler)

  FTM IP layer that sets the prescaler value for a given instance.
- boolean Ftm_Icu_Ip_GetOverflow (uint8 instance)

  FTM IP function that get the state of the overflow flag.
- Ftm_Icu_Ip_LevelType Ftm_Icu_Ip_GetInputLevel (uint8 instance, uint8 hwChannel)

  The API shall return the input read value for the selected pin, if hardware support the functionality.
- boolean Ftm_Icu_Ip_GetInputState (uint8 instance, uint8 hwChannel)

  Return input state of the channel.
- void Ftm_Icu_Ip_EnableInterrupt (uint8 instance, uint8 hwChannel)

  Enable channel interrupt.
- void Ftm_Icu_Ip_DisableInterrupt (uint8 instance, uint8 hwChannel)

  Disable channel interrupt.
- void Ftm_Icu_Ip_EnableNotification (uint8 instance, uint8 hwChannel)

  Driver function Enable Notification for timestamp.
- void Ftm_Icu_Ip_DisableNotification (uint8 instance, uint8 hwChannel)

  Driver function Disable Notification for timestamp.
- INTERRUPT_FUNC void FTM_0_ISR (void)

  Independent interrupt handler.
- INTERRUPT_FUNC void FTM_1_ISR (void)

  Independent interrupt handler.

6.2.2 Data Structure Documentation

6.2.2.1 struct Ftm_Icu_Ip_DutyCycleType

Structure that contains ICU Duty cycle parameters. It contains the values needed for calculating duty cycles i.e Period time value and active time value.

Definition at line 250 of file Ftm_Icu_Ip_Types.h.

Data Fields

Type	Name	Description
uint16	ActiveTime	Low or High time value.
uint16	PeriodTime	Period time value.

6.2.2.2 struct Ftm_Icu_Ip_ChannelConfigType

FlexTimer driver Input capture parameters for each channel.

Definition at line 263 of file Ftm_Icu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	hwChannel	Physical hardware channel ID
Ftm_Icu_Ip_ModeType	chMode	FlexTimer module mode of operation
Ftm_Icu_Ip_SubModeType	chSubMode	FlexTimer specific name of operation to execute
Ftm_Icu_Ip_MeasType	measurementMode	Measurement Mode for signal measurement
Ftm_Icu_Ip_EdgeType	edgeAlignement	Edge alignment Mode for signal measurement
boolean	continuouseEn	Continuous measurement state
uint8	filterValue	Filter Value
Ftm_Icu_Ip_CallbackType	callback	The callback function for channels edge detect events
uint8	callbackParams	The parameters of callback functions for channels events
Ftm_Icu_Ip_TimestampBufferType	timestampBufferType	Timestamp buffer type for timestamp mode
Ftm_Icu_Ip_LogicChState	logicChStateCallback	Store address of function used to change the logic state of the channel in HLD.
Ftm_Icu_Ip_NotifyType	ftmChannelNotification	The notification functions shall have no parameters and no return value.
Ftm_Icu_Ip_NotifyType	ftmOverflowNotification	The overflow notification functions shall have no parameters and no return value.

6.2.2.3 struct Ftm_Icu_Ip_InstanceConfigType

FTM IP layer module configuration.

Definition at line 283 of file Ftm_Icu_Ip_Types.h.

Data Fields

Type	Name	Description
Ftm_Icu_Ip_ClockSourceType	cfgClkSrc	Type of clock source used.
uint8	cfgPrescaler	Prescaler value.
uint8	cfgAltPrescaler	Alternant prescaler value.
Ftm_Icu_Ip_DebugModeType	debugMode	Debug mode.
uint16	maxCountValue	Maximum counter value. Minimum value is 0.

6.2.2.4 struct Ftm_Icu_Ip_ConfigType

FTM driver input capture parameters.

Definition at line 300 of file Ftm_Icu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	nNumChannels	Number of input capture channel used.
const Ftm_Icu_Ip_InstanceConfigType *	pInstanceConfig	Input capture instance configuration.
const Ftm_Icu_Ip_ChannelConfigType(*	pChannelsConfig)[]	Input capture channels configuration.

6.2.2.5 struct Ftm_Icu_Ip_ChStateType

This structure is used by the IPL driver for internal logic.

Definition at line 312 of file Ftm_Icu_Ip_Types.h.

Data Fields

Type	Name	Description
boolean	firstEdge	Store the status of the first measurement.
boolean	firstEdgePolarity	Store the first edge come to measurement in BOTH_EDGE mode.
Ftm_Icu_Ip_MeasType	measurement	Signal measurement mode.
uint16	ftm_Icu_Ip_aPeriod	Variable for saving the period.
uint16	ftm_Icu_Ip_aActivePulseWidth	Variable for saving the pulse width of active time.
Ftm_Icu_Ip_NotifyType	ftmChannelNotification	The notification functions for TIME_STAMP or SIGNAL_EDGE_DETECT mode.
Ftm_Icu_Ip_NotifyType	ftmOverflowNotification	The overflow notification functions.
uint16	edgeNumbers	Logic variable to count edges.

Data Fields

Type	Name	Description
Ftm_Icu_Ip_ModeType	channelMode	FTM channel mode.
Ftm_Icu_Ip_EdgeType	edgeTrigger	Type of edge used for activation.
Ftm_Icu_Ip_SubModeType	dmaMode	Support DMA or not.
uint16 *	ftm_Icu_Ip_aBuffer	Pointer to the buffer-array where the timestamp values shall be placed.
uint16	ftm_Icu_Ip_aBufferSize	Variable for saving the size of the external buffer (number of entries).
uint16	ftm_Icu_Ip_aBufferNotify	Variable for saving Notification interval (number of events).
uint16	ftm_Icu_Ip_aNotifyCount	Variable for saving the number of notify counts.
uint16	ftm_Icu_Ip_aBufferIndex	Variable for saving the time stamp index.
Ftm_Icu_Ip_TimestampBufferType	timestampBufferType	Timestamp buffer type for timestamp mode.
Ftm_Icu_Ip_LogicChState	logicChStateCallback	Store address of function used to change the logic state of the channel in HLD.
Ftm_Icu_Ip_CallbackType	callback	Calback for other types of measurement.
uint16	callbackParam	Logic channel for which callback is executed.
boolean	notificationEnable	Store the state determines whether Notifications are enabled or not.

6.2.2.6 struct Ftm_Icu_Ip_InstStateType

This structure is used by the IPL driver for internal logic.

Definition at line 364 of file Ftm_Icu_Ip_Types.h.

Data Fields

Type	Name	Description
boolean	instInit	Module initialization state.
uint8	prescaler	Module prescaler value.
uint8	prescalerAlternate	Module alternate prescaler value.
uint8	spuriousMask	

6.2.3 Macro Definition Documentation

6.2.3.1 CHAN0_IDX

```
#define CHAN0_IDX
```

Channel number for CHAN0.

Definition at line 97 of file Ftm_Icu_Ip_Types.h.

6.2.3.2 CHAN1_IDX

```
#define CHAN1_IDX
```

Channel number for CHAN1.

Definition at line 99 of file Ftm_Icu_Ip_Types.h.

6.2.3.3 CHAN2_IDX

```
#define CHAN2_IDX
```

Channel number for CHAN2.

Definition at line 101 of file Ftm_Icu_Ip_Types.h.

6.2.3.4 CHAN3_IDX

```
#define CHAN3_IDX
```

Channel number for CHAN3.

Definition at line 103 of file Ftm_Icu_Ip_Types.h.

6.2.3.5 CHAN4_IDX

```
#define CHAN4_IDX
```

Channel number for CHAN4.

Definition at line 105 of file Ftm_Icu_Ip_Types.h.

6.2.3.6 FTM_MAX_VAL_COUNTER

```
#define FTM_MAX_VAL_COUNTER
```

Maximum value of FTM counter.

Definition at line 108 of file Ftm_Icu_Ip_Types.h.

6.2.3.7 FTM_COMBINE_COMBINEx_SHIFT

```
#define FTM_COMBINE_COMBINEx_SHIFT(
             u8ChannelIdx )
```

Set flag COMINEx for specified pair: 0, 1, 2.

Definition at line 112 of file Ftm_Icu_Ip_Types.h.

6.2.4 Types Reference

6.2.4.1 Ftm_Icu_Ip_NotifyType

```
typedef void(* Ftm_Icu_Ip_NotifyType) (void)
```

The notification functions shall have no parameters and no return value.

Definition at line 245 of file Ftm_Icu_Ip_Types.h.

6.2.4.2 Ftm_Icu_Ip_CallbackType

```
typedef void(* Ftm_Icu_Ip_CallbackType) (uint16 callbackParam1, boolean callbackParam2)
```

Callback type for each channel.

Definition at line 257 of file Ftm_Icu_Ip_Types.h.

6.2.4.3 Ftm_Icu_Ip_LogicChState

```
typedef void(* Ftm_Icu_Ip_LogicChState) (uint16 logicChannel, uint8 mask, boolean set)
```

Callback type for each channel.

Definition at line 260 of file Ftm_Icu_Ip_Types.h.

6.2.5 Enum Reference

6.2.5.1 Ftm_Icu_Ip_ClockSourceType

```
enum Ftm_Icu_Ip_ClockSourceType
```

FTM clock source selection.

Enumerator

FTM_NO_CLOCK_SELECTED	No clock selected. This in effect disables the FTM counter.
FTM_SYSTEM_CLOCK	FTM input clock.
FTM_FIXED_FREQUENCY_CLOCK	Fixed frequency clock.
FTM_EXTERNAL_CLOCK	External clock.

Definition at line 120 of file Ftm_Icu_Ip_Types.h.

6.2.5.2 Ftm_Icu_Ip_DebugModeType

enum Ftm_Icu_Ip_DebugModeType

FTM debug modes of operation.

Enumerator

MODE_0	FTM counter - stopped.
MODE_1	FTM counter - stopped.
MODE_2	FTM counter - stopped.
MODE_3	FTM counter - functional mode.

Definition at line 133 of file Ftm_Icu_Ip_Types.h.

6.2.5.3 Ftm_Icu_Ip_EdgeType

enum Ftm_Icu_Ip_EdgeType

Activation condition for the measurement - selecting edge type.

Enumerator

FTM_ICU_NO_PIN_CONTROL	No trigger.
FTM_ICU_RISING_EDGE	Rising edge trigger.
FTM_ICU_FALLING_EDGE	Falling edge trigger.
FTM_ICU_BOTH_EDGES	Rising and falling edge trigger.

Definition at line 146 of file Ftm_Icu_Ip_Types.h.

6.2.5.4 Ftm_Icu_Ip_ModeType

enum Ftm_Icu_Ip_ModeType

Operation mode for ICU driver.

Enumerator

FTM_ICU_MODE_NO_MEASUREMENT	No measurement mode.
FTM_ICU_MODE_SIGNAL_EDGE_DETECT	Signal edge detect measurement mode.
FTM_ICU_MODE_SIGNAL_MEASUREMENT	Signal measurement mode.
FTM_ICU_MODE_TIMESTAMP	Timestamp measurement mode.
FTM_ICU_MODE_EDGE_COUNTER	Edge counter measurement mode.

Definition at line 159 of file Ftm_Icu_Ip_Types.h.

6.2.5.5 Ftm_Icu_Ip_SubModeType

enum Ftm_Icu_Ip_SubModeType

Enable/disable DMA support for timestamp.

Definition at line 174 of file Ftm_Icu_Ip_Types.h.

6.2.5.6 Ftm_Icu_Ip_MeasType

enum Ftm_Icu_Ip_MeasType

Type of operation for signal measurement.

Enumerator

FTM_ICU_NO_MEASUREMENT	No measurement.
FTM_ICU_LOW_TIME	The time measurement for OFF period.
FTM_ICU_HIGH_TIME	The time measurement for ON period.
FTM_ICU_PERIOD_TIME	Period measurement between two consecutive falling/raising edges.
FTM_ICU_DUTY_CYCLE	The fraction of active period.

Definition at line 183 of file Ftm_Icu_Ip_Types.h.

6.2.5.7 Ftm_Icu_Ip_LevelType

enum Ftm_Icu_Ip_LevelType

Enumeration used for returning the level of input pin.

Enumerator

FTM_ICU_LEVEL_LOW	Low level state.
FTM_ICU_LEVEL_HIGH	High level state.

Definition at line 200 of file Ftm_Icu_Ip_Types.h.

6.2.5.8 Ftm_Icu_Ip_ClockModeType

enum Ftm_Icu_Ip_ClockModeType

Definition of prescaler type (Normal or Alternate)

Enumerator

FTM_ICU_NORMAL_CLK	Normal prescaler
FTM_ICU_ALTERNATE_CLK	Alternate prescaler

Definition at line 211 of file Ftm_Icu_Ip_Types.h.

6.2.5.9 Ftm_Icu_Ip_StatusType

enum Ftm_Icu_Ip_StatusType

Generic error codes.

Enumerator

FTM_IP_STATUS_SUCCESS	Generic operation success status.
FTM_IP_STATUS_ERROR	Generic operation failure status.

Definition at line 219 of file Ftm_Icu_Ip_Types.h.

6.2.5.10 Ftm_Icu_Ip_TimestampBufferType

```
enum Ftm_Icu_Ip_TimestampBufferType
```

Type of operation for timestamp.

Enumerator

FTM_ICU_NO_TIMESTAMP	No timestamp.
FTM_ICU_CIRCULAR_BUFFER	The timestamp with circular buffer .
FTM_ICU_LINEAR_BUFFER	The timestamp with linear buffer .

Definition at line 230 of file Ftm_Icu_Ip_Types.h.

6.2.6 Function Reference

6.2.6.1 Ftm_Icu_Ip_Init()

```
Ftm_Icu_Ip_StatusType Ftm_Icu_Ip_Init (
            uint8 instance,
            const Ftm_Icu_Ip_ConfigType * userConfig )
```

This function configures the channel in the Input Capture mode for either getting time-stamps on edge detection or on signal measurement. When the edge specified in the captureMode argument occurs on the channel and then the FTM counter is captured into the CnV register. The user have to read the CnV register separately to get this value. The filter function is disabled if the filterVal argument passed as 0. The filter feature. is available only on channels 0,1,2,3.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*userConfig*	Configuration of the input capture channel.

Returns

Ftm_Icu_Ip_StatusType

- FTM_IP_STATUS_SUCCESS : Completed successfully.
- FTM_IP_STATUS_ERROR : Error occurred.

6.2.6.2 Ftm_Icu_Ip_DeInit()

```
Ftm_Icu_Ip_StatusType Ftm_Icu_Ip_DeInit (
            uint8 instance )
```

Disables input capture mode and clears FTM timer configuration.

Parameters

in	*instance*	Hardware instance of FTM used.

6.2.6.3 Ftm_Icu_Ip_SetSleepMode()

```
void Ftm_Icu_Ip_SetSleepMode (
            uint8 instance,
            uint8 hwChannel )
```

Driver function sets FTM hardware channel into SLEEP mode.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void

6.2.6.4 Ftm_Icu_Ip_SetNormalMode()

```
void Ftm_Icu_Ip_SetNormalMode (
            uint8 instance,
            uint8 hwChannel )
```

Driver function sets FTM hardware channel into NORMAL mode.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void

6.2.6.5 Ftm_Icu_Ip_GetCaptureRegisterValue()

```
uint32 Ftm_Icu_Ip_GetCaptureRegisterValue (
            uint8 instance,
            uint8 hwChannel )
```

Capture the value of counter register for a specified channel.

The API shall return the value stored in capture register. The API is the equivalent of AUTOSAR API Get↩ CaptureRegisterValue.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

uint32 Value of the register captured.

6.2.6.6 Ftm_Icu_Ip_SetActivationCondition()

```
void Ftm_Icu_Ip_SetActivationCondition (
            uint8 instance,
            uint8 hwChannel,
            Ftm_Icu_Ip_EdgeType activation )
```

This function enables the requested activation condition(rising, falling or both edges) for corresponding FTM channels.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.
in	*activation*	Edge activation type used.

Returns

void

6.2.6.7 Ftm_Icu_Ip_StartTimestamp()

```
void Ftm_Icu_Ip_StartTimestamp (
            uint8 instance,
            uint8 hwChannel,
            Ftm_Icu_ValueType * bufferPtr,
            uint16 bufferSize,
            uint16 notifyInterval )
```

FTM IP layer function which starts timestamp measure mode.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.
in	*bufferPtr*	buffer pointer for results
in	*bufferSize*	size of buffer results
in	*notifyInterval*	interval for calling notification

Returns

void

6.2.6.8 Ftm_Icu_Ip_StopTimestamp()

```
void Ftm_Icu_Ip_StopTimestamp (
            uint8 instance,
            uint8 hwchannel )
```

FTM IP layer function which stops timestamp measure mode for a given instance and channel.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void

6.2.6.9 Ftm_Icu_Ip_GetTimestampIndex()

```
uint16 Ftm_Icu_Ip_GetTimestampIndex (
            uint8 instance,
            uint8 hwChannel )
```

This function reads the timestamp index of the given channel.

The API shall return the index to be used for next timestamp measurement to be stored.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*channel*	Hardware channel of FTM used.

Returns

uint16 - Timestamp index of the given channel, next index to be used for storring timestamps.

Precondition

Ftm_Icu_Ip_Init must be called before. Icu_StartTimestamp must be called before.

6.2.6.10 Ftm_Icu_Ip_EnableEdgeDetection()

```
void Ftm_Icu_Ip_EnableEdgeDetection (
            uint8 instance,
            uint8 hwChannel )
```

FTM IP layer function which enable edge detection measure mode for a given instance and channel.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

Ftm_Icu_Ip_StatusType

6.2.6.11 Ftm_Icu_Ip_DisableEdgeDetection()

```
void Ftm_Icu_Ip_DisableEdgeDetection (
            uint8 instance,
            uint8 hwChannel )
```

FTM IP layer function which disable edge detection measure mode for a given instance and channel.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

Ftm_Icu_Ip_StatusType

6.2.6.12 Ftm_Icu_Ip_ResetEdgeCount()

```
void Ftm_Icu_Ip_ResetEdgeCount (
            uint8 instance,
            uint8 hwchannel )
```

FTM IP layer function which reset edge count measure mode for a given instance and channel.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void

6.2.6.13 Ftm_Icu_Ip_EnableEdgeCount()

```
Ftm_Icu_Ip_StatusType Ftm_Icu_Ip_EnableEdgeCount (
            uint8 instance,
            uint8 hwChannel )
```

FTM IP layer function which enable edge count measure mode for a given instance and channel.

Parameters

in	*instance*	module instance number
in	*hwChannel*	Hardware channel of FTM used.

Returns

Ftm_Icu_Ip_StatusType

6.2.6.14 Ftm_Icu_Ip_DisableEdgeCount()

```
void Ftm_Icu_Ip_DisableEdgeCount (
            uint8 instance,
            uint8 hwChannel )
```

FTM IP layer function which disable edge count measure mode for a given instance and channel.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

Ftm_Icu_Ip_StatusType

6.2.6.15 Ftm_Icu_Ip_GetEdgeNumbers()

```
uint16 Ftm_Icu_Ip_GetEdgeNumbers (
            uint8 instance,
            uint8 hwChannel )
```

FTM IP layer function which gets the number of edges for a given instance and channel.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

uint16 Number of edges counted.

6.2.6.16 Ftm_Icu_Ip_StartSignalMeasurement()

```
void Ftm_Icu_Ip_StartSignalMeasurement (
            uint8 instance,
            uint8 hwchannel )
```

FTM IP layer function which starts signal measurement mode for a given instance and channel.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void

6.2.6.17 Ftm_Icu_Ip_StopSignalMeasurement()

```
void Ftm_Icu_Ip_StopSignalMeasurement (
            uint8 instance,
            uint8 hwchannel )
```

FTM IP layer function which stops signal measurement mode for a given instance and channel.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void

6.2.6.18 Ftm_Icu_Ip_GetTimeElapsed()

```
uint16 Ftm_Icu_Ip_GetTimeElapsed (
            uint8 instance,
            uint8 hwChannel )
```

This function reads the elapsed Signal Low, High or Period Time for the given channel.

This service is reentrant and reads the elapsed Signal Low Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Low Time. The elapsed time is measured between a falling edge and the consecutive rising edge of the channel. This service reads the elapsed Signal High Time for the given channel that is configured in Measurement Mode Signal Measurement,Signal High Time.The elapsed time is measured between a rising edge and the consecutive falling edge of the channel. This service reads the elapsed Signal Period Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Period Time.The elapsed time is measured between consecutive rising (or falling) edges of the channel. The period start edge is

configurable.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

uint16 - the elapsed Signal Low Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Low Time

Precondition

Ftm_Icu_Ip_Init must be called before. The channel must be configured in Measurement Mode Signal Measurement.

6.2.6.19 Ftm_Icu_Ip_GetDutyCycleValues()

```
void Ftm_Icu_Ip_GetDutyCycleValues (
            uint8 instance,
            uint8 hwChannel,
            Ftm_Icu_Ip_DutyCycleType * dutyCycleValues )
```

This function reads the coherent active time and period time for the given ICU Channel.

The function is reentrant and reads the coherent active time and period time for the given ICU Channel, if it is configured in Measurement Mode Signal Measurement, Duty Cycle Values.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.
out	*dutyCycleValues*	structure where duty cycle values are stored

Returns

void

Precondition

Ftm_Icu_Ip_Init must be called before. The given channel must be configured in Measurement Mode Signal Measurement, Duty Cycle Values.

6.2.6.20 Ftm_Icu_Ip_SetPrescaler()

```
void Ftm_Icu_Ip_SetPrescaler (
            uint8 instance,
            Ftm_Icu_Ip_ClockModeType selectPrescaler )
```

FTM IP layer that sets the prescaler value for a given instance.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*prescaler*	Value of the prescaler.

6.2.6.21 Ftm_Icu_Ip_GetOverflow()

```
boolean Ftm_Icu_Ip_GetOverflow (
            uint8 instance )
```

FTM IP function that get the state of the overflow flag.

Parameters

in	*instance*	Hardware instance of FTM used.

Returns

boolean State of the overflow flag.

Return values

TRUE	The overflow flag is set.
FALSE	The overflow flag is not set.

6.2.6.22 Ftm_Icu_Ip_GetInputLevel()

```
Ftm_Icu_Ip_LevelType Ftm_Icu_Ip_GetInputLevel (
            uint8 instance,
            uint8 hwChannel )
```

The API shall return the input read value for the selected pin, if hardware support the functionality.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

Ftm_Icu_Ip_LevelType - Level detected at this time on the input pin

6.2.6.23 Ftm_Icu_Ip_GetInputState()

```
boolean Ftm_Icu_Ip_GetInputState (
            uint8 instance,
            uint8 hwChannel )
```

Return input state of the channel.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

boolean Channel level type.

6.2.6.24 Ftm_Icu_Ip_EnableInterrupt()

```
void Ftm_Icu_Ip_EnableInterrupt (
             uint8 instance,
             uint8 hwChannel )
```

Enable channel interrupt.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void.

6.2.6.25 Ftm_Icu_Ip_DisableInterrupt()

```
void Ftm_Icu_Ip_DisableInterrupt (
             uint8 instance,
             uint8 hwChannel )
```

Disable channel interrupt.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void.

6.2.6.26 Ftm_Icu_Ip_EnableNotification()

```
void Ftm_Icu_Ip_EnableNotification (
            uint8 instance,
            uint8 hwChannel )
```

Driver function Enable Notification for timestamp.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void

6.2.6.27 Ftm_Icu_Ip_DisableNotification()

```
void Ftm_Icu_Ip_DisableNotification (
            uint8 instance,
            uint8 hwChannel )
```

Driver function Disable Notification for timestamp.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void

6.2.6.28 FTM_0_ISR()

```
INTERRUPT_FUNC void FTM_0_ISR (
            void  )
```

Independent interrupt handler.

Interrupt handler for FTM module 0.

6.2.6.29 FTM_1_ISR()

```
INTERRUPT_FUNC void FTM_1_ISR (
             void  )
```

Independent interrupt handler.

Interrupt handler for FTM module 1.

6.3 SIUL2 IPL

6.3.1 Detailed Description SIUL2 HW module.

SIUL2 IP layer hardware module.

SIUL2 provides control over all electrical pin controls and ports with 16 bits of bidirectional, general-purpose input and output signals. SIUL2 enables you to select the functions and electrical characteristics that appear on external chip pins. It also controls the multiplexing of internal signals from one module to another and controls chip I/O. It supports as many as 32 external interrupts with trigger event configuration.

SIUL2 provides dedicated pad control to general-purpose pads that can be configured as either inputs or outputs. It provides registers for you to read values from GPIO pads configured as inputs and to write values to GPIO pads configured as outputs:

- When configured as output, you can write to an internal register to control the state driven on the associated output pad.
- When configured as **input**, you can detect the state of the associated pad by reading the value from an internal register.
- When configured as input and output, the pad value can be read back to check if the written value appeared on the pad.

Data Structures

- struct Siul2_Icu_Ip_ChannelConfigType
  SIUL2 IP layer channel configuration structure. *More...*
- struct Siul2_Icu_Ip_InstanceConfigType
  SIUL2 IP layer instance configuration structure. *More...*
- struct Siul2_Icu_Ip_State
  SIUL2 IP state structure. *More...*
- struct Siul2_Icu_Ip_ConfigType
  SIUL2 IP layer configuration structure. *More...*

Macros

- #define ICU_START_SEC_CODE
  SIUL2 External Interrupt Channels defines.

Types Reference

- typedef void(* Siul2_Icu_Ip_NotifyType) (void)
  The notification functions shall have no parameters and no return value.
- typedef void(* Siul2_Icu_Ip_CallbackType) (uint16 callbackParam1, boolean callbackParam2)
  Callback signature used in each channel with an active interrupt.

Enum Reference

- enum Siul2_Icu_Ip_ClockModeType
  Definition of prescaler type.
- enum Siul2_Icu_Ip_EdgeType
  Siul2_Icu_ActivationType.
- enum Siul2_Icu_Ip_IrqDmaSelectType
  Siul2_Icu_IrqDmaSelectType.
- enum Siul2_Icu_Ip_StatusType
  SIUL2 IP layer operation status.

Function Reference

- Siul2_Icu_Ip_StatusType Siul2_Icu_Ip_DeInit (uint8 instance)
  Driver function that de-initializes SIUL hardware channel.
- Siul2_Icu_Ip_StatusType Siul2_Icu_Ip_Init (uint8 instance, const Siul2_Icu_Ip_ConfigType *userConfig)
  Driver function that initializes SIUL hardware channel.
- void Siul2_Icu_Ip_SetActivationCondition (uint8 instance, uint8 hwChannel, Siul2_Icu_Ip_EdgeType edge)
- boolean Siul2_Icu_Ip_GetInputState (uint8 instance, uint8 hwChannel)
  ICU driver function that sets activation condition of SIUL2 channel.
- void Siul2_Icu_Ip_EnableInterrupt (uint8 instance, uint8 hwChannel)
  ICU driver function that enables the interrupt of SIUL2 channel.
- void Siul2_Icu_Ip_DisableInterrupt (uint8 instance, uint8 hwChannel)
  ICU driver function that disables the interrupt of SIUL2 channel.
- void Siul2_Icu_Ip_SetSleepMode (uint8 instance, uint8 hwChannel)
  Driver function sets SIUL2 hardware channel into SLEEP mode.
- void Siul2_Icu_Ip_SetNormalMode (uint8 instance, uint8 HwChannel)
  Driver function that sets SIUL2 hardware channel into NORMAL mode.
- void Siul2_Icu_Ip_SetClockMode (uint8 instance, Siul2_Icu_Ip_ClockModeType mode)
  Icu driver function used to set the global prescaler of a SIUL2 module.
- void Siul2_Icu_Ip_EnableNotification (uint8 instance, uint8 hwChannel)
  Driver function Enable Notification for timestamp.
- void Siul2_Icu_Ip_DisableNotification (uint8 instance, uint8 hwChannel)
  Driver function Disable Notification for timestamp.
- INTERRUPT_FUNC void SIUL2_1_ICU_EIRQ_SINGLE_INT_HANDLER (void)
  Interrupt handler for SIUL2 instance 1.
- void Siul2_Icu_SetUserAccessAllowed (uint32 siul2BaseAddr)
  Siul2_Icu_SetUserAccessAllowed.

6.3.2 Data Structure Documentation

6.3.2.1 struct Siul2_Icu_Ip_ChannelConfigType

SIUL2 IP layer channel configuration structure.

Definition at line 149 of file Siul2_Icu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	hwChannel	The interrupt pin index
boolean	digFilterEn	Enables digital filter
uint8	maxFilterCnt	Maximum interrupt filter value
Siul2_Icu_Ip_IrqDmaSelectType	intSel	Switch between DMA and interrupt request
Siul2_Icu_Ip_EdgeType	intEdgeSel	The type of edge event
Siul2_Icu_Ip_CallbackType	callback	Pointer to the callback function.
Siul2_Icu_Ip_NotifyType	Siul2ChannelNotification	The notification functions shall have no parameters and no return value.
uint8	callbackParam	The logic channel for which callback is set.

6.3.2.2 struct Siul2_Icu_Ip_InstanceConfigType

SIUL2 IP layer instance configuration structure.

Definition at line 162 of file Siul2_Icu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	intFilterClk	Siul2 interrupt clock prescaller digital filter.
uint8	altIntFilterClk	Siul2 interrupt clock prescaller digital filter.

6.3.2.3 struct Siul2_Icu_Ip_State

SIUL2 IP state structure.

This structure is used by the IPL driver for internal logic. The content is populated at initialization time.

Definition at line 173 of file Siul2_Icu_Ip_Types.h.

Data Fields

Type	Name	Description
boolean	chInit	Initialization state.
Siul2_Icu_Ip_CallbackType	callback	Pointer to the callback function.
Siul2_Icu_Ip_NotifyType	Siul2ChannelNotification	The notification functions for SIGNAL_EDGE_DETECT mode.
uint16	callbackParam	The logic channel for which callback is set.
boolean	notificationEnable	State of the notification.

6.3.2.4 struct Siul2_Icu_Ip_ConfigType

SIUL2 IP layer configuration structure.

Definition at line 184 of file Siul2_Icu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	numChannels	Number of channels in the configuration.
const Siul2_Icu_Ip_InstanceConfigType *	pInstanceConfig	Pointer to the instance configuration.
const Siul2_Icu_Ip_ChannelConfigType(*	pChannelsConfig)[]	Pointer to the channels configuration.

6.3.3 Macro Definition Documentation

6.3.3.1 ICU_START_SEC_CODE

```
#define ICU_START_SEC_CODE
```

SIUL2 External Interrupt Channels defines.

Definition at line 171 of file Siul2_Icu_Ip_Irq.h.

6.3.4 Types Reference

6.3.4.1 Siul2_Icu_Ip_NotifyType

```
typedef void(* Siul2_Icu_Ip_NotifyType) (void)
```

The notification functions shall have no parameters and no return value.

Definition at line 144 of file Siul2_Icu_Ip_Types.h.

6.3.4.2 Siul2_Icu_Ip_CallbackType

```
typedef void(* Siul2_Icu_Ip_CallbackType) (uint16 callbackParam1, boolean callbackParam2)
```

Callback signature used in each channel with an active interrupt.

Definition at line 146 of file Siul2_Icu_Ip_Types.h.

6.3.5 Enum Reference

6.3.5.1 Siul2_Icu_Ip_ClockModeType

```
enum Siul2_Icu_Ip_ClockModeType
```

Definition of prescaler type.

Definition of prescaler type (Normal or Alternate)

Enumerator

SIUL2_ICU_NORMAL_CLK	Normal prescaler
SIUL2_ICU_ALTERNATE_CLK	Alternate prescaler

Definition at line 104 of file Siul2_Icu_Ip_Types.h.

6.3.5.2 Siul2_Icu_Ip_EdgeType

enum Siul2_Icu_Ip_EdgeType

Siul2_Icu_ActivationType.

This indicates the activation type SIUL2 channel (Rising, Falling or Both)

Enumerator

SIUL2_ICU_DISABLE	Interrupt disable.
SIUL2_ICU_RISING_EDGE	Interrupt on rising edge.
SIUL2_ICU_FALLING_EDGE	Interrupt on falling edge.
SIUL2_ICU_BOTH_EDGES	Interrupt on either edge.

Definition at line 115 of file Siul2_Icu_Ip_Types.h.

6.3.5.3 Siul2_Icu_Ip_IrqDmaSelectType

enum Siul2_Icu_Ip_IrqDmaSelectType

Siul2_Icu_IrqDmaSelectType.

This indicates the type of request DMA or IRQ when activation edge is detected

Enumerator

SIUL2_ICU_IRQ	Generate an interrupt request.
SIUL2_ICU_DMA	Generate an DMA request

Definition at line 127 of file Siul2_Icu_Ip_Types.h.

6.3.5.4 Siul2_Icu_Ip_StatusType

```
enum Siul2_Icu_Ip_StatusType
```

SIUL2 IP layer operation status.

Enumerator

SIUL2_ICU_IP_SUCCESS	Status for success operation return.
SIUL2_ICU_IP_ERROR	General error return status.

Definition at line 134 of file Siul2_Icu_Ip_Types.h.

6.3.6 Function Reference

6.3.6.1 Siul2_Icu_Ip_DeInit()

```
Siul2_Icu_Ip_StatusType Siul2_Icu_Ip_DeInit (
            uint8 instance )
```

Driver function that de-initializes SIUL hardware channel.

This function:

- Restore to reset values SIUL2 registers used on init.

Parameters

in	*instance*	- Instance number used

Returns

Siul2_Icu_Ip_StatusType - The status of DeInit

6.3.6.2 Siul2_Icu_Ip_Init()

```
Siul2_Icu_Ip_StatusType Siul2_Icu_Ip_Init (
            uint8 instance,
            const Siul2_Icu_Ip_ConfigType * userConfig )
```

Driver function that initializes SIUL hardware channel.

This function:

- Disables interrupt.
- Sets Interrupt filter enable register
- Sets Interrupt Filter Clock Prescaler Register
- Sets Activation Condition

Parameters

in	*instance*	Hardware instance of SIUL2 used.
in	*userConfig*	Instance configuration.

Returns

Siul2_Icu_Ip_StatusType - The status of Init

6.3.6.3 Siul2_Icu_Ip_SetActivationCondition()

```
void Siul2_Icu_Ip_SetActivationCondition (
            uint8 instance,
            uint8 hwChannel,
            Siul2_Icu_Ip_EdgeType edge )
```

This function enables the requested activation condition(rising, falling or both edges) for corresponding SIUL2 channels.

Parameters

in	*instance*	Hardware instance of SIUL2 used.
in	*hwChannel*	Hardware channel of SIUL2 used.
in	*edge*	Edge activation type used.

6.3.6.4 Siul2_Icu_Ip_GetInputState()

```
boolean Siul2_Icu_Ip_GetInputState (
            uint8 instance,
            uint8 hwChannel )
```

ICU driver function that sets activation condition of SIUL2 channel.

Parameters

in	*instance*	Hardware instance of SIUL2 used.
in	*hwChannel*	Hardware channel of SIUL2 used.

Returns

boolean Input state.

6.3.6.5 Siul2_Icu_Ip_EnableInterrupt()

```
void Siul2_Icu_Ip_EnableInterrupt (
             uint8 instance,
             uint8 hwChannel )
```

ICU driver function that enables the interrupt of SIUL2 channel.

This function enables SIUL2 Channel Interrupt.

Parameters

in	*instance*	Hardware instance of SIUL2 used.
in	*hwChannel*	Hardware channel of SIUL2 used.

Returns

void

6.3.6.6 Siul2_Icu_Ip_DisableInterrupt()

```
void Siul2_Icu_Ip_DisableInterrupt (
             uint8 instance,
             uint8 hwChannel )
```

ICU driver function that disables the interrupt of SIUL2 channel.

This function disables SIUL2 Channel Interrupt.

Parameters

in	*instance*	Hardware instance of SIUL2 used.
in	*hwChannel*	Hardware channel of SIUL2 used.

Returns

void

6.3.6.7 Siul2_Icu_Ip_SetSleepMode()

```
void Siul2_Icu_Ip_SetSleepMode (
             uint8 instance,
             uint8 hwChannel )
```

Driver function sets SIUL2 hardware channel into SLEEP mode.

This function enables the interrupt if wakeup is enabled for corresponding SIUL2 channel.

Parameters

in	*instance*	Hardware instance of SIUL2 used.
in	*hwChannel*	Hardware channel of SIUL2 used.

Returns

void

6.3.6.8 Siul2_Icu_Ip_SetNormalMode()

```
void Siul2_Icu_Ip_SetNormalMode (
             uint8 instance,
             uint8 HwChannel )
```

Driver function that sets SIUL2 hardware channel into NORMAL mode.

This function enables the interrupt if Notification is enabled for corresponding SIUL2 channel.

Parameters

in	*instance*	Hardware instance of SIUL2 used.
in	*hwChannel*	Hardware channel of SIUL2 used.

Returns

void

6.3.6.9 Siul2_Icu_Ip_SetClockMode()

```
void Siul2_Icu_Ip_SetClockMode (
            uint8 instance,
            Siul2_Icu_Ip_ClockModeType mode )
```

Icu driver function used to set the global prescaler of a SIUL2 module.

This function:

- Sets IFCPR register with a prescaler value

Parameters

in	*instance*	Hardware instance of SIUL2 used.
in	*mode*	Global prescaler for the SIUL2 module.

Returns

void

6.3.6.10 Siul2_Icu_Ip_EnableNotification()

```
void Siul2_Icu_Ip_EnableNotification (
            uint8 instance,
            uint8 hwChannel )
```

Driver function Enable Notification for timestamp.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void

6.3.6.11 Siul2_Icu_Ip_DisableNotification()

```
void Siul2_Icu_Ip_DisableNotification (
            uint8 instance,
            uint8 hwChannel )
```

Driver function Disable Notification for timestamp.

Parameters

in	*instance*	Hardware instance of FTM used.
in	*hwChannel*	Hardware channel of FTM used.

Returns

void

6.3.6.12 SIUL2_1_ICU_EIRQ_SINGLE_INT_HANDLER()

```
INTERRUPT_FUNC void SIUL2_1_ICU_EIRQ_SINGLE_INT_HANDLER (
            void  )
```

Interrupt handler for SIUL2 instance 1.

Process the interrupt of SIUL2 instance 1. @isr

Note

This will be defined only if the single interrupt mode is configured.

6.3.6.13 Siul2_Icu_SetUserAccessAllowed()

```
void Siul2_Icu_SetUserAccessAllowed (
            uint32 siul2BaseAddr )
```

Siul2_Icu_SetUserAccessAllowed.

This function is called externally by OS Application

Parameters

in	*siul2BaseAddr*	- The base address of Siul2 module.

6.4 FTM

6.4.1 Detailed Description

Function Reference

- void Ftm_Icu_SetUserAccessAllowed (uint32 FtmBaseAddr)
  Ftm_Icu_SetUserAccessAllowed.

6.4.2 Function Reference

6.4.2.1 Ftm_Icu_SetUserAccessAllowed()

```
void Ftm_Icu_SetUserAccessAllowed (
             uint32 FtmBaseAddr )
```

Ftm_Icu_SetUserAccessAllowed.

This function is called externally by OS Application

Parameters

in	*FtmBaseAddr*	- The base address of Ftm module.

6.5 Icu Driver

6.5.1 Detailed Description

Data Structures

- struct Icu_ChannelConfigType

  Structure that contains ICU channel configuration. More...
- struct Icu_ConfigType

  This type contains initialization data. More...
- struct Icu_DutyCycleType

  Structure that contains ICU Duty cycle parameters. It contains the values needed for calculating duty cycles i.e Period time value and active time value. More...

Macros

- #define ICU_STOPTIMESTAMP_ID

  API service ID for Icu_StopTimestamp function.
- #define ICU_E_NOT_STARTED

  API service Icu_StopTimestamp called on a channel which was not started or already stopped.

Types Reference

- typedef uint8 Icu_ChannelStateType

  ICU Channel state type.
- typedef uint16 Icu_ChannelType

  This gives the numeric ID (hardware channel number) of an ICU channel.
- typedef Icu_TimerRegisterWidthType Icu_ValueType

  Type for saving the timer register width value.
- typedef Icu_HwSpecificIndexType Icu_IndexType

  Type for saving the ICU Hardware specific index.
- typedef Icu_HwSpecificEdgeNumberType Icu_EdgeNumberType

  Type for saving hardware specific edge number.
- typedef uint32 Icu_WakeupValueType

  Type for saving the Wakeup value.
- typedef uint16 Icu_MeasurementSubModeType

  Type for saving the ICU measurement submode type.
- typedef void(* Icu_NotifyType) (void)

  The notification functions shall have no parameters and no return value.

Enum Reference

- enum Icu_ModeType

  Allow enabling or disabling of all interrupts which are not required for the ECU wakeup.
- enum Icu_InputStateType

  Input state of an ICU channel.
- enum Icu_MeasurementModeType

  Definition of the measurement mode type.
- enum Icu_SignalMeasurementPropertyType

  Definition of the measurement property type.
- enum Icu_TimestampBufferType

  Definition of the timestamp measurement property type.
- enum Icu_ActivationType

  Definition of the type of activation of an ICU channel.
- enum Icu_LevelType

  Return the status of the pin.
- enum Icu_SelectPrescalerType

  Definition of prescaler type.

Function Reference

- void Icu_Init (const Icu_ConfigType *ConfigPtr)

  This function initializes the driver.
- void Icu_DeInit (void)

  This function de-initializes the ICU module.
- void Icu_SetMode (Icu_ModeType Mode)

  This function sets the ICU mode.
- void Icu_DisableWakeup (Icu_ChannelType Channel)

  This function disables the wakeup capability of a single ICU channel.
- void Icu_EnableWakeup (Icu_ChannelType Channel)

  This function (re-)enables the wakeup capability of the given ICU channel.
- void Icu_CheckWakeup (EcuM_WakeupSourceType WakeupSource)

  Checks if a wakeup capable ICU channel is the source for a wakeup event.
- void Icu_SetActivationCondition (Icu_ChannelType Channel, Icu_ActivationType Activation)

  This function sets the activation-edge for the given channel.
- void Icu_DisableNotification (Icu_ChannelType Channel)

  This function disables the notification of a channel.
- void Icu_EnableNotification (Icu_ChannelType Channel)

  This function enables the notification on the given channel.
- Icu_InputStateType Icu_GetInputState (Icu_ChannelType Channel)

  This function returns the status of the ICU input.
- void Icu_StartTimestamp (Icu_ChannelType Channel, Icu_ValueType *BufferPtr, uint16 BufferSize, uint16 NotifyInterval)

  This function starts the capturing of timer values on the edges.
- void Icu_StopTimestamp (Icu_ChannelType Channel)

  This function stops the timestamp measurement of the given channel.

- Icu_IndexType Icu_GetTimestampIndex (Icu_ChannelType Channel)

  This function reads the elapsed Signal Low, High or Period Time for the given channel.
- void Icu_ResetEdgeCount (Icu_ChannelType Channel)

  This function resets the value of the counted edges to zero.
- void Icu_EnableEdgeCount (Icu_ChannelType Channel)

  This function enables the counting of edges of the given channel.
- void Icu_DisableEdgeCount (Icu_ChannelType Channel)

  This function disables the counting of edges of the given channel.
- Icu_EdgeNumberType Icu_GetEdgeNumbers (Icu_ChannelType Channel)

  This function reads the number of counted edges.
- void Icu_EnableEdgeDetection (Icu_ChannelType Channel)

  This function enables or re-enables the detection of edges of the given channel.
- void Icu_DisableEdgeDetection (Icu_ChannelType Channel)

  This function disables the detection of edges of the given channel.
- Icu_ValueType Icu_GetTimeElapsed (Icu_ChannelType Channel)

  This function reads the elapsed Signal Low, High or Period Time for the given channel.
- void Icu_GetDutyCycleValues (Icu_ChannelType Channel, Icu_DutyCycleType *DutyCycleValues)

  This function reads the coherent active time and period time for the given ICU Channel.
- void Icu_StartSignalMeasurement (Icu_ChannelType Channel)

  This function starts the measurement of signals.
- void Icu_StopSignalMeasurement (Icu_ChannelType Channel)

  This function stops the measurement of signals of the given channel.
- void Icu_GetVersionInfo (Std_VersionInfoType *versioninfo)

  This service returns the version information of this module.
- void Icu_SetClockMode (Icu_SelectPrescalerType selectPrescaler)

  This function sets all channels prescalers based on the input mode.
- Icu_LevelType Icu_GetInputLevel (Icu_ChannelType Channel)

  This function returns the actual status of PIN.
- Icu_ValueType Icu_GetCaptureRegisterValue (Icu_ChannelType Channel)

  This function starts the measurement of signals.
- void Icu_ReportWakeupAndOverflow (uint16 Channel, boolean bOverflow)

  This function reports the wakeup and overflow events, if available.
- void Icu_ReportEvents (uint16 Channel, boolean bOverflow)

  This function reports the wakeup event, overflow event and notification, if available.
- void Icu_LogicChStateCallback (uint16 logicChannel, uint8 mask, boolean set)

  Signature of change logic channel state callback function.

Variables

- const Icu_ConfigType * Icu_pCfgPtr [(1U)]

  Pointer initialized during init with the address of the received configuration structure.
- Icu_ModeType Icu_CurrentMode

  Saves the current Icu mode.
- volatile Icu_ChannelStateType Icu_aChannelState [((Icu_ChannelType) 12U)]

  Stores actual state and configuration of ICU Channels.
- Icu_ValueType * Icu_aBuffer [((Icu_ChannelType) 12U)]

Pointer to the buffer-array where the timestamp values shall be placed.

- Icu_ValueType Icu_aBufferSize [((Icu_ChannelType) 12U)]

  Array for saving the size of the external buffer (number of entries)
- Icu_ValueType Icu_aBufferNotify [((Icu_ChannelType) 12U)]

  Array for saving Notification interval (number of events).
- volatile Icu_ValueType Icu_aNotifyCount [((Icu_ChannelType) 12U)]

  Array for saving the number of notify counts.
- volatile Icu_IndexType Icu_aBufferIndex [((Icu_ChannelType) 12U)]

  Array for saving the time stamp index.

6.5.2 Data Structure Documentation

6.5.2.1 struct Icu_ChannelConfigType

Structure that contains ICU channel configuration.

It contains the information like Icu Channel Mode, Channel Notification function, overflow Notification function.

Definition at line 537 of file Icu.h.

Data Fields

- boolean Icu_WakeupCapabile

  Channel wakeup capability enable.
- Icu_ActivationType Icu_ActivEdge

  RISING_EDGE, FALLING_EDGE or BOTH_EDGES for EDGE_COUNTER.
- Icu_MeasurementModeType Icu_ChannelMode

  EDGE_DETECT, TIME_STAMP, SIGNAL_MEASUREMENT or EDGE_COUNTER.
- Icu_MeasurementSubModeType Icu_ChannelProperty

  CIRCULAR_BUFFER or LINEAR_BUFFER for TIME_STAMP, DUTY_CYCLE, HIGH_TIME, LOW_TIME or PERIOD_TIME for SIGNAL_MEASUREMENT and RISING_EDGE, FALLING_EDGE or BOTH_EDGES for EDGE_COUNTER.
- Icu_NotifyType Icu_ChannelNotification

  Icu Channel Notification function for TIME_STAMP or EDGE_COUNTER mode.
- Icu_WakeupValueType Icu_Channel_WakeupValue

  EcuM wakeup source Id.
- const Icu_Ipw_ChannelConfigType * Icu_IpwChannelConfigPtr

  Pointer to the ipw channel pointer configuration.

6.5.2.1.1 Field Documentation

6.5.2.1.1.1 Icu_WakeupCapabile `boolean Icu_WakeupCapabile`

Channel wakeup capability enable.

Definition at line 540 of file Icu.h.

6.5.2.1.1.2 Icu_ActivEdge Icu_ActivationType Icu_ActivEdge

RISING_EDGE, FALLING_EDGE or BOTH_EDGES for EDGE_COUNTER.

Definition at line 542 of file Icu.h.

6.5.2.1.1.3 Icu_ChannelMode Icu_MeasurementModeType Icu_ChannelMode

EDGE_DETECT, TIME_STAMP, SIGNAL_MEASUREMENT or EDGE_COUNTER.

Definition at line 544 of file Icu.h.

6.5.2.1.1.4 Icu_ChannelProperty Icu_MeasurementSubModeType Icu_ChannelProperty

CIRCULAR_BUFFER or LINEAR_BUFFER for TIME_STAMP, DUTY_CYCLE, HIGH_TIME, LOW_TIME or PERIOD_TIME for SIGNAL_MEASUREMENT and RISING_EDGE, FALLING_EDGE or BOTH_EDGES for EDGE_COUNTER.

Definition at line 548 of file Icu.h.

6.5.2.1.1.5 Icu_ChannelNotification Icu_NotifyType Icu_ChannelNotification

Icu Channel Notification function for TIME_STAMP or EDGE_COUNTER mode.

Definition at line 550 of file Icu.h.

6.5.2.1.1.6 Icu_Channel_WakeupValue Icu_WakeupValueType Icu_Channel_WakeupValue

EcuM wakeup source Id.

Definition at line 560 of file Icu.h.

6.5.2.1.1.7 Icu_IpwChannelConfigPtr const Icu_Ipw_ChannelConfigType* Icu_IpwChannelConfigPtr

Pointer to the ipw channel pointer configuration.

Definition at line 563 of file Icu.h.

6.5.2.2 struct Icu_ConfigType

This type contains initialization data.

he notification functions shall be configurable as function pointers within the initialization data structure (Icu_ConfigType). This type of the external data structure shall contain the initialization data for the ICU driver. It shall contain:

- Wakeup Module Info (in case the wakeup-capability is true)
- ICU dependent properties for used HW units
- Clock source with optional prescaler (if provided by HW)

Definition at line 577 of file Icu.h.

Data Fields

- uint8 nNumChannels
  The number of configured logical channels.
- const Icu_ChannelConfigType(* Icu_ChannelConfigPtr)[]
  Pointer to the list of Icu configured channels.
- uint8 nNumInstances
  The number of IP instances configured.
- const Icu_Ipw_IpConfigType(* Icu_IpConfigPtr)[]
  Pointer to the list of Icu configured channels.
- const uint8(* Icu_IndexChannelMap)[]
  channel index in each partition map table
- uint8 u32CoreId
  Core index.

6.5.2.2.1 Field Documentation

6.5.2.2.1.1 nNumChannels `uint8 nNumChannels`

The number of configured logical channels.

Definition at line 580 of file Icu.h.

6.5.2.2.1.2 Icu_ChannelConfigPtr `const Icu_ChannelConfigType(* Icu_ChannelConfigPtr)[]`

Pointer to the list of Icu configured channels.

Definition at line 583 of file Icu.h.

6.5.2.2.1.3 nNumInstances `uint8 nNumInstances`

The number of IP instances configured.

Definition at line 586 of file Icu.h.

6.5.2.2.1.4 Icu_IpConfigPtr `const Icu_Ipw_IpConfigType(* Icu_IpConfigPtr)[]`

Pointer to the list of Icu configured channels.

Definition at line 589 of file Icu.h.

6.5.2.2.1.5 Icu_IndexChannelMap `const uint8(* Icu_IndexChannelMap)[]`

channel index in each partition map table

Definition at line 592 of file Icu.h.

6.5.2.2.1.6 u32CoreId `uint8 u32CoreId`

Core index.

Definition at line 595 of file Icu.h.

6.5.2.3 struct Icu_DutyCycleType

Structure that contains ICU Duty cycle parameters. It contains the values needed for calculating duty cycles i.e Period time value and active time value.

Definition at line 271 of file Icu_Types.h.

Data Fields

- Icu_ValueType ActiveTime
  Low or High time value.
- Icu_ValueType PeriodTime
  Period time value.

6.5.2.3.1 Field Documentation

6.5.2.3.1.1 ActiveTime Icu_ValueType ActiveTime

Low or High time value.

Definition at line 273 of file Icu_Types.h.

6.5.2.3.1.2 PeriodTime Icu_ValueType PeriodTime

Period time value.

Definition at line 274 of file Icu_Types.h.

6.5.3 Macro Definition Documentation

6.5.3.1 ICU_STOPTIMESTAMP_ID

```
#define ICU_STOPTIMESTAMP_ID
```

API service ID for Icu_StopTimestamp function.

Parameters used when raising an error/exception

Definition at line 507 of file Icu.h.

6.5.3.2 ICU_E_NOT_STARTED

```
#define ICU_E_NOT_STARTED
```

API service Icu_StopTimestamp called on a channel which was not started or already stopped.

Definition at line 514 of file Icu.h.

6.5.4 Types Reference

6.5.4.1 Icu_ChannelStateType

```
typedef uint8 Icu_ChannelStateType
```

ICU Channel state type.

Definition at line 220 of file Icu_Types.h.

6.5.4.2 Icu_ChannelType

```
typedef uint16 Icu_ChannelType
```

This gives the numeric ID (hardware channel number) of an ICU channel.

Definition at line 225 of file Icu_Types.h.

6.5.4.3 Icu_ValueType

```
typedef Icu_TimerRegisterWidthType Icu_ValueType
```

Type for saving the timer register width value.

Definition at line 230 of file Icu_Types.h.

6.5.4.4 Icu_IndexType

```
typedef Icu_HwSpecificIndexType Icu_IndexType
```

Type for saving the ICU Hardware specific index.

Definition at line 237 of file Icu_Types.h.

6.5.4.5 Icu_EdgeNumberType

```
typedef Icu_HwSpecificEdgeNumberType Icu_EdgeNumberType
```

Type for saving hardware specific edge number.

Definition at line 244 of file Icu_Types.h.

6.5.4.6 Icu_WakeupValueType

```
typedef uint32 Icu_WakeupValueType
```

Type for saving the Wakeup value.

Definition at line 251 of file Icu_Types.h.

6.5.4.7 Icu_MeasurementSubModeType

```
typedef uint16 Icu_MeasurementSubModeType
```

Type for saving the ICU measurement submode type.

Definition at line 258 of file Icu_Types.h.

6.5.4.8 Icu_NotifyType

```
typedef void(* Icu_NotifyType) (void)
```

The notification functions shall have no parameters and no return value.

Definition at line 263 of file Icu_Types.h.

6.5.5 Enum Reference

6.5.5.1 Icu_ModeType

```
enum Icu_ModeType
```

Allow enabling or disabling of all interrupts which are not required for the ECU wakeup.

Enumerator

ICU_MODE_NORMAL	Normal operation, all used interrupts are enabled according to the notification requests.
ICU_MODE_SLEEP	Reduced power operation. In sleep mode only those notifications are available which are configured as wakeup capable.

Definition at line 102 of file Icu_Types.h.

6.5.5.2 Icu_InputStateType

```
enum Icu_InputStateType
```

Input state of an ICU channel.

Enumerator

ICU_IDLE	No activation edge has been detected since the last call of Icu_GetInputState() or Icu_Init().
ICU_ACTIVE	An activation edge has been detected.

Definition at line 116 of file Icu_Types.h.

6.5.5.3 Icu_MeasurementModeType

`enum Icu_MeasurementModeType`

Definition of the measurement mode type.

Enumerator

ICU_MODE_SIGNAL_EDGE_DETECT	Mode for detecting edges.
ICU_MODE_SIGNAL_MEASUREMENT	Mode for measuring different times between various configurable edges.
ICU_MODE_TIMESTAMP	Mode for capturing timer values on configurable edges.
ICU_MODE_EDGE_COUNTER	Mode for counting edges on configurable edges.

Definition at line 128 of file Icu_Types.h.

6.5.5.4 Icu_SignalMeasurementPropertyType

`enum Icu_SignalMeasurementPropertyType`

Definition of the measurement property type.

Enumerator

ICU_LOW_TIME	The channel is configured for reading the elapsed Signal Low Time.
ICU_HIGH_TIME	The channel is configured for reading the elapsed Signal High Time.
ICU_PERIOD_TIME	The channel is configured for reading the elapsed Signal Period Time.
ICU_DUTY_CYCLE	The channel is configured to read values which are needed for calculating the duty cycle (coherent Active and Period Time).

Definition at line 145 of file Icu_Types.h.

6.5.5.5 Icu_TimestampBufferType

enum Icu_TimestampBufferType

Definition of the timestamp measurement property type.

Enumerator

ICU_LINEAR_BUFFER	The buffer will just be filled once.
ICU_CIRCULAR_BUFFER	After reaching the end of the buffer, the driver restarts at the beginning of the buffer.

Definition at line 164 of file Icu_Types.h.

6.5.5.6 Icu_ActivationType

enum Icu_ActivationType

Definition of the type of activation of an ICU channel.

Enumerator

ICU_RISING_EDGE	An appropriate action shall be executed when a rising edge occurs on the ICU input signal.
ICU_FALLING_EDGE	An appropriate action shall be executed when a falling edge occurs on the ICU input signal.
ICU_BOTH_EDGES	An appropriate action shall be executed when either a rising or falling edge occur on the ICU input signal.

Definition at line 175 of file Icu_Types.h.

6.5.5.7 Icu_LevelType

enum Icu_LevelType

Return the status of the pin.

Enumeration of to check the status of pin.

Enumerator

ICU_LEVEL_LOW	Default Input PIN Status.
ICU_LEVEL_HIGH	As Icu_GetInputState do not give the Actual PIN status user can call the Non Autosar API Icu_GetInputLevel to get the Actual status of PIN.

Definition at line 191 of file Icu_Types.h.

6.5.5.8 Icu_SelectPrescalerType

```
enum Icu_SelectPrescalerType
```

Definition of prescaler type.

Enumerator

ICU_NORMAL_CLOCK_MODE	Default channel prescaler.
ICU_ALTERNATE_CLOCK_MODE	Alternate channel prescaler mode.

Definition at line 208 of file Icu_Types.h.

6.5.6 Function Reference

6.5.6.1 Icu_Init()

```
void Icu_Init (
            const Icu_ConfigType * ConfigPtr )
```

This function initializes the driver.

This service is a non reentrant function used for driver initialization. The Initialization function shall initialize all relevant registers of the configured hardware with the values of the structure referenced by the parameter Config↩ Ptr. If the hardware allows for only one usage of the register, the driver module implementing that functionality is responsible for initializing the register. The initialization function of this module shall always have a pointer as a parameter, even though for Variant PC no configuration set shall be given. Instead a NULL pointer shall be passed to the initialization function. The Icu module environment shall not call Icu_Init during a running operation (e. g. timestamp measurement or edge counting).

Parameters

in	*ConfigPtr*	Pointer to a selected configuration structure.

Returns

void

6.5.6.2 Icu_DeInit()

```
void Icu_DeInit (
            void  )
```

This function de-initializes the ICU module.

This service is a Non reentrant function used for ICU De-Initialization After the call of this service, the state of the peripherals used by configuration shall be the same as after power on reset. Values of registers which are not writable are excluded. This service shall disable all used interrupts and notifications. The Icu module environment shall not call Icu_DeInit during a running operation (e. g. timestamp measurement or edge counting)

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.3 Icu_SetMode()

```
void Icu_SetMode (
            Icu_ModeType Mode )
```

This function sets the ICU mode.

This service is a non reentrant function used for ICU mode selection. This service shall set the operation mode to the given mode parameter. This service can be called during running operations. If so, an ongoing operation that generates interrupts on a wakeup capable channel like e.g. time stamping or edge counting might lead to the ICU module not being able to properly enter sleep mode. This is then a system or ECU configuration issue not a problem of this specification.

Parameters

in	*Mode*	Specifies the operation mode

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.4 Icu_DisableWakeup()

```
void Icu_DisableWakeup (
            Icu_ChannelType Channel )
```

This function disables the wakeup capability of a single ICU channel.

This service is reentrant function and shall disable the wakeup capability of a single ICU channel.This service is only feasible for ICU channels configured statically as wakeup capable true. The function Icu_DisableWakeup shall be pre compile time configurable On,Off by the configuration parameter IcuDisableWakeupApi.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.5 Icu_EnableWakeup()

```
void Icu_EnableWakeup (
            Icu_ChannelType Channel )
```

This function (re-)enables the wakeup capability of the given ICU channel.

The function is reentrant and re-enable the wake-up capability of a single ICU channel.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured as wakeup capable.

6.5.6.6 Icu_CheckWakeup()

```
void Icu_CheckWakeup (
            EcuM_WakeupSourceType WakeupSource )
```

Checks if a wakeup capable ICU channel is the source for a wakeup event.

The function calls the ECU state manager service EcuM_SetWakeupEvent in case of a valid ICU channel wakeup event.

Parameters

in	*WakeupSource*	Information on wakeup source to be checked.

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured as wakeup capable.

6.5.6.7 Icu_SetActivationCondition()

```
void Icu_SetActivationCondition (
            Icu_ChannelType Channel,
            Icu_ActivationType Activation )
```

This function sets the activation-edge for the given channel.

This service is reentrant and shall set the activation-edge according to Activation parameter for the given channel. This service shall support channels which are configured for the following Icu_MeasurementMode:

- ICU_MODE_SIGNAL_EDGE_DETECT
- ICU_MODE_TIMESTAMP
- ICU_MODE_EDGE_COUNTER

Parameters

in	*Channel*	Logical number of the ICU channel
in	*Activation*	Type of activation.

Returns

void

Precondition

Icu_Init must be called before. The channel must be properly configured (ICU_MODE_SIGNAL_EDGE↩ _DETECT, ICU_MODE_TIMESTAMP, ICU_MODE_EDGE_COUNTER).

6.5.6.8 Icu_DisableNotification()

```
void Icu_DisableNotification (
            Icu_ChannelType Channel )
```

This function disables the notification of a channel.

This function is reentrant and disables the notification of a channel.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.9 Icu_EnableNotification()

```
void Icu_EnableNotification (
            Icu_ChannelType Channel )
```

This function enables the notification on the given channel.

This function is reentrant and enables the notification on the given channel. The notification will be reported only when the channel measurement property is enabled or started

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.10 Icu_GetInputState()

```
Icu_InputStateType Icu_GetInputState (
            Icu_ChannelType Channel )
```

This function returns the status of the ICU input.

This service is reentrant shall return the status of the ICU input. Only channels which are configured for the following Icu_MeasurementMode shall be supported:

- ICU_MODE_SIGNAL_EDGE_DETECT,
- ICU_MODE_SIGNAL_MEASUREMENT.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

Icu_InputStateType

Return values

ICU_ACTIVE	An activation edge has been detected
ICU_IDLE	No activation edge has been detected since the last call of Icu_GetInputState() or Icu_Init().

Precondition

Icu_Init must be called before.

6.5.6.11 Icu_StartTimestamp()

```
void Icu_StartTimestamp (
            Icu_ChannelType Channel,
            Icu_ValueType * BufferPtr,
            uint16 BufferSize,
            uint16 NotifyInterval )
```

This function starts the capturing of timer values on the edges.

This function is reentrant and starts the capturing of timer values on the edges activated by the service Icu_SetActivationCondition() to an external buffer.

Parameters

in	*Channel*	Logical number of the ICU channel
in	*BufferPtr*	Pointer to the buffer-array where the timestamp values shall be placed.
in	*BufferSize*	Size of the external buffer (number of entries)
in	*NotifyInterval*	Notification interval (number of events).

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.12 Icu_StopTimestamp()

```
void Icu_StopTimestamp (
            Icu_ChannelType Channel )
```

This function stops the timestamp measurement of the given channel.

This function is reentrant and stops the timestamp measurement of the given channel.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.13 Icu_GetTimestampIndex()

```
Icu_IndexType Icu_GetTimestampIndex (
            Icu_ChannelType Channel )
```

This function reads the elapsed Signal Low, High or Period Time for the given channel.

This service is reentrant and reads the elapsed Signal Low Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Low Time. The elapsed time is measured between a falling edge and the consecutive rising edge of the channel. This service reads the elapsed Signal High Time for the given channel that is configured in Measurement Mode Signal Measurement,Signal High Time.The elapsed time is measured between a rising edge and the consecutive falling edge of the channel. This service reads the elapsed Signal Period Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Period Time.The elapsed time is measured between consecutive rising (or falling) edges of the channel. The period start edge is

configurable.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

Icu_ValueType - the elapsed Signal Low Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Low Time

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Signal Measurement.

6.5.6.14 Icu_ResetEdgeCount()

```
void Icu_ResetEdgeCount (
            Icu_ChannelType Channel )
```

This function resets the value of the counted edges to zero.

This function is reentrant and resets the value of the counted edges to zero.

Parameters

in	*Channel*	Logical number of the ICU channel.

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.15 Icu_EnableEdgeCount()

```
void Icu_EnableEdgeCount (
            Icu_ChannelType Channel )
```

This function enables the counting of edges of the given channel.

This service is reentrant and shall enable the counting of edges of the given channel. Note: This service doesnot do the real counting itself. This is done by the hardware (capture unit). Only the configured edges shall be counted (rising edge, falling edge or both edges).

Configuration of the edge is done in Icu_Init or Icu_SetActivationCondition.The configured edge can be changed during runtime using Icu_SetActivationCondition. Interrupts are not required for edge counting.If interrupts are enabled, the interrupt service routine will set the overflow flag if more than 0xFFFFFF edges are measured.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before. The given channel must be configured in Measurement Mode Edge Counter.

6.5.6.16 Icu_DisableEdgeCount()

```
void Icu_DisableEdgeCount (
            Icu_ChannelType Channel )
```

This function disables the counting of edges of the given channel.

This function is reentrant and disables the counting of edges of the given channel.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before.The given channel must be configured in Measurement Mode Edge Counter.

6.5.6.17 Icu_GetEdgeNumbers()

```
Icu_EdgeNumberType Icu_GetEdgeNumbers (
            Icu_ChannelType Channel )
```

This function reads the number of counted edges.

This function is reentrant reads the number of counted edges after the last call of Icu_ResetEdgeCount().

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

Icu_EdgeNumberType - Number of the counted edges.

Precondition

Icu_Init must be called before. The given channel must be configured in Measurement Mode Edge Counter.

6.5.6.18 Icu_EnableEdgeDetection()

```
void Icu_EnableEdgeDetection (
            Icu_ChannelType Channel )
```

This function enables or re-enables the detection of edges of the given channel.

This function is reentrant enables or re-enables the detection of edges of the given channel.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Edge Counter

6.5.6.19 Icu_DisableEdgeDetection()

```
void Icu_DisableEdgeDetection (
            Icu_ChannelType Channel )
```

This function disables the detection of edges of the given channel.

This function is reentrant and disables the detection of edges of the given channel.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Edge Detection.

6.5.6.20 Icu_GetTimeElapsed()

```
Icu_ValueType Icu_GetTimeElapsed (
            Icu_ChannelType Channel )
```

This function reads the elapsed Signal Low, High or Period Time for the given channel.

This service is reentrant and reads the elapsed Signal Low Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Low Time. The elapsed time is measured between a falling edge and the consecutive rising edge of the channel. This service reads the elapsed Signal High Time for the given channel that is configured in Measurement Mode Signal Measurement,Signal High Time.The elapsed time is measured between a rising edge and the consecutive falling edge of the channel. This service reads the elapsed Signal Period Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Period Time.The elapsed time is measured between consecutive rising (or falling) edges of the channel. The period start edge is

configurable.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

Icu_ValueType - the elapsed Signal Low Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Low Time

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Signal Measurement.

6.5.6.21 Icu_GetDutyCycleValues()

```
void Icu_GetDutyCycleValues (
            Icu_ChannelType Channel,
            Icu_DutyCycleType * DutyCycleValues )
```

This function reads the coherent active time and period time for the given ICU Channel.

The function is reentrant and reads the coherent active time and period time for the given ICU Channel, if it is configured in Measurement Mode Signal Measurement, Duty Cycle Values.

Parameters

in	*Channel*	Logical number of the ICU channel
out	*DutyCycleValues*	Pointer to a buffer where the results (high time and period time) shall be placed.

Returns

void

Precondition

Icu_Init must be called before. The given channel must be configured in Measurement Mode Signal Measurement, Duty Cycle Values.

6.5.6.22 Icu_StartSignalMeasurement()

```
void Icu_StartSignalMeasurement (
            Icu_ChannelType Channel )
```

This function starts the measurement of signals.

This service is reentrant and starts the measurement of signals beginning with the configured default start edge which occurs first after the call of this service. This service shall only be available in Measurement Mode ICU_M↩ ODE_SIGNAL_MEASUREMENT. This service shall reset the state for the given channel to ICU_IDLE.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Signal Measurement.

6.5.6.23 Icu_StopSignalMeasurement()

```
void Icu_StopSignalMeasurement (
            Icu_ChannelType Channel )
```

This function stops the measurement of signals of the given channel.

This function is reentrant and stops the measurement of signals of the given channel.

Parameters

in	*Channel*	- Logical number of the ICU channel

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Signal Measurement.

6.5.6.24 Icu_GetVersionInfo()

```
void Icu_GetVersionInfo (
            Std_VersionInfoType * versioninfo )
```

This service returns the version information of this module.

This service is Non reentrant and returns the version information of this module. The version information includes:

- Module Id
- Vendor Id
- Vendor specific version numbers If source code for caller and callee of this function is available this function should be realized as a macro. The macro should be defined in the modules header file.

Parameters

out	*versioninfo*	Pointer to location to store version info

Returns

void

6.5.6.25 Icu_SetClockMode()

```
void Icu_SetClockMode (
            Icu_SelectPrescalerType selectPrescaler )
```

This function sets all channels prescalers based on the input mode.

Parameters

selectPrescaler	Select the used prescaler: prescaler/alternatePresclaer.

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.26 Icu_GetInputLevel()

```
Icu_LevelType Icu_GetInputLevel (
            Icu_ChannelType Channel )
```

This function returns the actual status of PIN.

This function returns the actual status of PIN.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

Icu_LevelType

Precondition

Icu_Init must be called before.

6.5.6.27 Icu_GetCaptureRegisterValue()

```
Icu_ValueType Icu_GetCaptureRegisterValue (
            Icu_ChannelType Channel )
```

This function starts the measurement of signals.

. This service returns the value of Capture register. This API is used to measure the time difference between 2 different timer channels.

Parameters

in	*Channel*	Logical number of the ICU channel

Returns

Icu_ValueType Value of Capture register

Precondition

Icu_Init must be called before.

The given channel must be configured in SignalMeasurement or in Timestamp mode

6.5.6.28 Icu_ReportWakeupAndOverflow()

```
void Icu_ReportWakeupAndOverflow (
            uint16 Channel,
            boolean bOverflow )
```

This function reports the wakeup and overflow events, if available.

This function reports the wakeup and overflow events, if available. Called from hardware interrupt routine and route to user overflow handler

Parameters

in	*Channel*	Hardware number identifier of the ICU channel
in	*bOverflow*	Parameter that indicates the source of report is an overflow

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.29 Icu_ReportEvents()

```
void Icu_ReportEvents (
            uint16 Channel,
            boolean bOverflow )
```

This function reports the wakeup event, overflow event and notification, if available.

This function reports the wakeup event, overflow event and notification, if available

Parameters

in	*Channel*	Harware number identifier of the ICU channel
in	*overflow*	Parameter that indicates the source of report is an overflow

Returns

void

Precondition

Icu_Init must be called before.

6.5.6.30 Icu_LogicChStateCallback()

```
void Icu_LogicChStateCallback (
            uint16 logicChannel,
            uint8 mask,
            boolean set )
```

Signature of change logic channel state callback function.

Parameters

logicChannel	Logical number of the ICU channel
mask	Bit mark
set	Set value

6.5.7 Variable Documentation

6.5.7.1 Icu_pCfgPtr

```
const Icu_ConfigType* Icu_pCfgPtr[(1U)]  [extern]
```

Pointer initialized during init with the address of the received configuration structure.

Will be used by all functions to access the configuration data.

6.5.7.2 Icu_CurrentMode

```
Icu_ModeType Icu_CurrentMode  [extern]
```

Saves the current Icu mode.

6.5.7.3 Icu_aChannelState

```
volatile Icu_ChannelStateType Icu_aChannelState[((Icu_ChannelType) 12U)]  [extern]
```

Stores actual state and configuration of ICU Channels.

6.5.7.4 Icu_aBuffer

```
Icu_ValueType* Icu_aBuffer[((Icu_ChannelType) 12U)]  [extern]
```

Pointer to the buffer-array where the timestamp values shall be placed.

6.5.7.5 Icu_aBufferSize

```
Icu_ValueType Icu_aBufferSize[((Icu_ChannelType) 12U)]  [extern]
```

Array for saving the size of the external buffer (number of entries)

6.5.7.6 Icu_aBufferNotify

```
Icu_ValueType Icu_aBufferNotify[((Icu_ChannelType) 12U)]  [extern]
```

Array for saving Notification interval (number of events).

6.5.7.7 Icu_aNotifyCount

```
volatile Icu_ValueType Icu_aNotifyCount[((Icu_ChannelType) 12U)]  [extern]
```

Array for saving the number of notify counts.

6.5.7.8 Icu_aBufferIndex

```
volatile Icu_IndexType Icu_aBufferIndex[((Icu_ChannelType) 12U)]  [extern]
```

Array for saving the time stamp index.

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.